

Capítulo 3:

Estructuras de datos lineales y no lineales

Laura Viviana Galvis

Escuela de Ingeniería de Sistemas e Informática
Facultad de Ingenierías Fisicomecánicas
Universidad Industrial de Santander

2025-2

Escuela de
Ingeniería de
Sistemas e
Informática



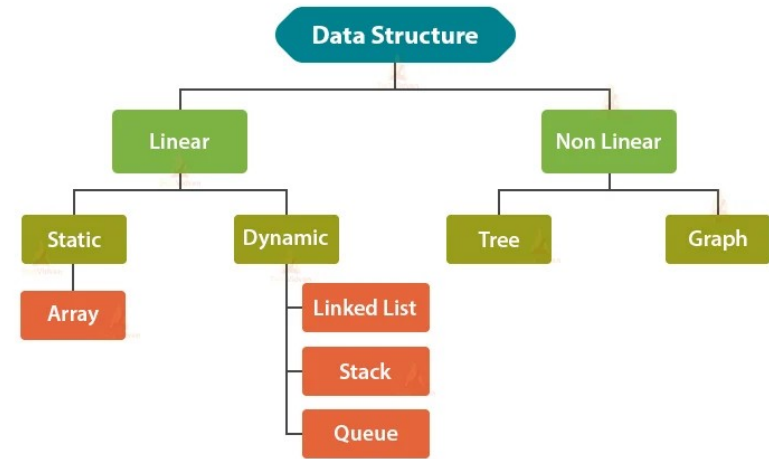
Universidad
Industrial de
Santander



Contenidos del capítulo

1. Estructuras de datos lineales

- a. Arrays
- b. Listas enlazadas
- c. Pilas
- d. Colas

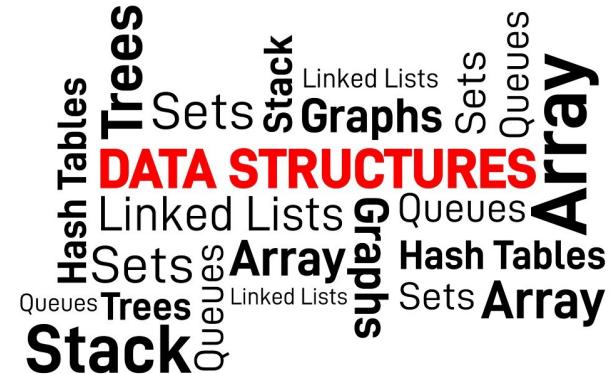


2. Estructuras de datos no lineales

- a. Árboles (binarios y balanceados)
- b. Grafos
- c. Montículo (heap)

3. De acceso directo:

- 4. a. Mapas, diccionarios y tablas hash



1. Estructuras de datos lineales

1.1 Listas Enlazadas

- Estructura de datos que representa una secuencia de **nodos**, conectados a través de **enlaces**
- Cada nodo está dividido en 2 (simple) o 3 partes (doble)
 - Simple: Donde se almacena el dato, y la dirección del siguiente nodo
 - Doble: Donde se almacena el dato, la dirección del siguiente nodo, y la dirección del anterior nodo
- Los enlaces tienen terminación NULL (excepto circulares)
 - Cuando el puntero al siguiente nodo es NULL, la lista ha llegado al final

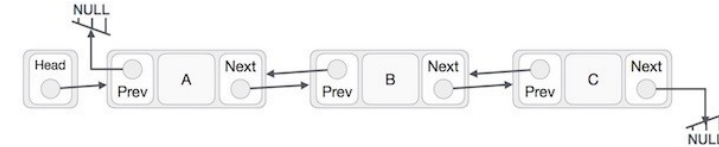
Aplicaciones:

- Gestión de la lista de canciones en reproductor multimedia.
 - Inicio/final, agregar canción, escuchar anterior/siguiente
- Combinaciones de teclado para *shortcuts*
- Historial de navegación
- Operaciones “hacer”, “deshacer”
- Una lista circular se usa en los videojuegos multijugador para controlar el orden de los jugadores.

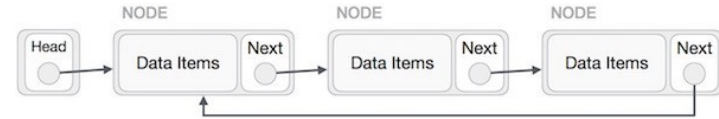
Lista enlazada simple



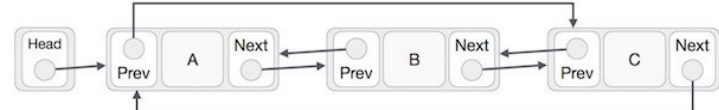
Lista enlazada doble



Lista enlazada circular (no tiene fin)



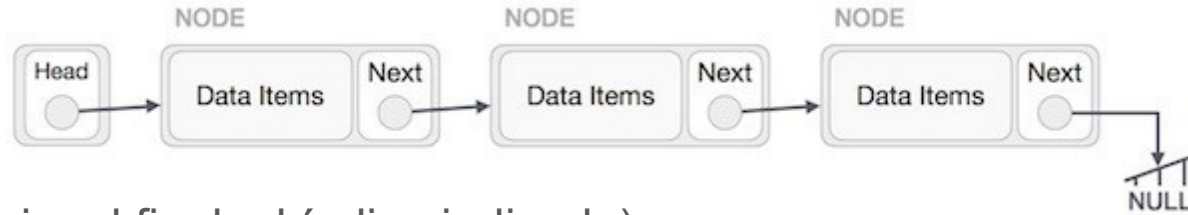
Lista enlazada circular doble



1.1 Lista Enlazada Simple

Estructura: Nodo (con **datos** y el **puntero** al siguiente)

Operaciones permitidas:



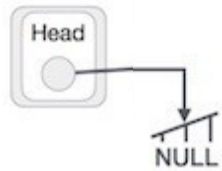
- **Creación**
- **Insertar** elementos (al inicio, al final, al índice indicado)
- **Recorrer** la lista para imprimirla
- Verificar si está **vacía**
- **Buscar** elementos
- **Eliminar** elementos (al inicio, al final, al índice indicado)

Construyendo un Nodo (**DIY**)



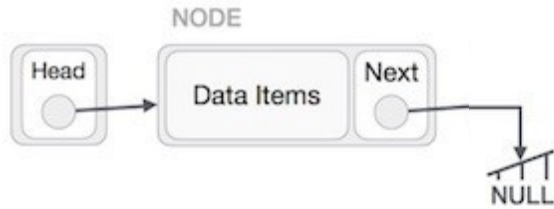
```
1  public class Nodo {
2
3      public int dato;
4      public Nodo siguiente = null;
5
6      public Nodo(int dato){
7          this.dato = dato;
8      }
9
10 }
```

Construyendo una Lista (**DIY**)



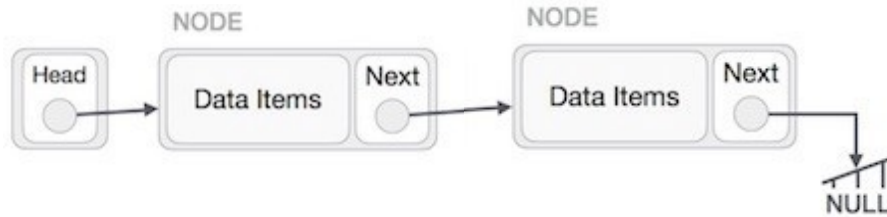
```
1  public class Lista {  
2  
3      private Nodo cabeza;  
4  
5      public Lista(){  
6          this.cabeza = null;  
7      }  
8  }
```

Insertando elementos **al inicio de** una Lista (**DIY**)



```
1  public void insertarNodoInicio(int dato){
2
3      Nodo nodoIni = new Nodo(dato);
4      nodoIni.siguiente = cabeza;
5      cabeza = nodoIni;
6  }
```

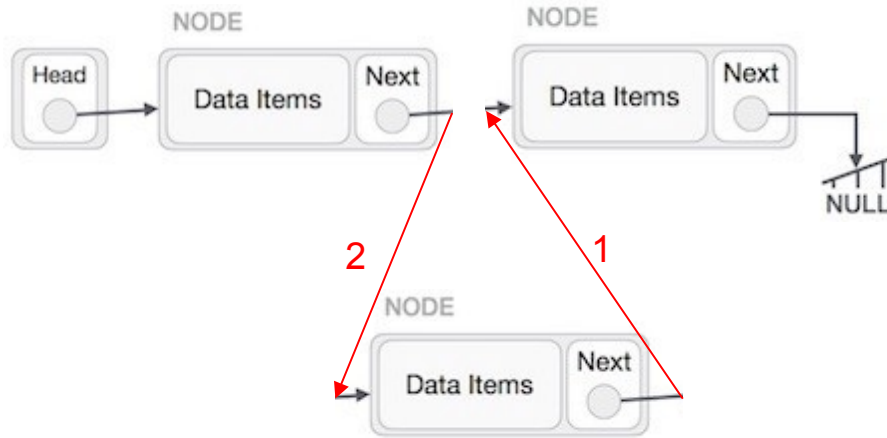

Insertando elementos **al final de** una Lista (**DIY**)



```

1  public void insertarNodoFinal(int dato){
2
3      Nodo nodoFin = new Nodo(dato);
4      Nodo nodoRecorre = cabeza;
5
6      while (nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8      }
9
10     nodoRecorre.siguiente = nodoFin;
11 }
  
```

Insertando elementos **al índice indicado** de una Lista (**DIY**)



```

1  public void insertarNodoIndice(int dato, int idx){
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < idx && nodoRecorre.siguiete != null){
7          nodoRecorre = nodoRecorre.siguiete;
8          cont++;
9      }
10
11     nodoIndice.siguiete = nodoRecorre.siguiete;
12     nodoRecorre.siguiete = nodoIndice;
13
14 }
    
```



Imprimiendo elementos de una Lista (**DIY**)

```
1 public void imprimirLista(){
2     Nodo nodoRecorre = cabeza;
3
4     while (nodoRecorre != null){
5         System.out.print(nodoRecorre.dato + " -> ");
6         nodoRecorre = nodoRecorre.siguiente;
7     }
8     System.out.println("NULL");
9 }
```

```
1 run:
2 NULL
3 10 -> NULL
4 5 -> 10 -> NULL
5 1 -> 5 -> 10 -> NULL
6 1 -> 5 -> 10 -> 20 -> NULL
7 1 -> 5 -> 10 -> 20 -> 30 -> NULL
8 1 -> 5 -> 10 -> 15 -> 20 -> 30 -> NULL
9 1 -> 3 -> 5 -> 10 -> 15 -> 20 -> 30 -> NULL
```

Probando...



```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Insertando al inicio");
8      listaEjemplo.insertarNodoInicio(10);
9      listaEjemplo.imprimirLista();
10
11     listaEjemplo.insertarNodoInicio(5);
12     listaEjemplo.imprimirLista();
13
14     listaEjemplo.insertarNodoInicio(1);
15     listaEjemplo.imprimirLista();
16
17     System.out.println();
18     System.out.println("Insertando al final");
19     // Insertando al final
20     listaEjemplo.insertarNodoFinal(20);
21     listaEjemplo.imprimirLista();
22     listaEjemplo.insertarNodoFinal(30);
23     listaEjemplo.imprimirLista();
24
25     System.out.println();
26     System.out.println("Insertando en índice");
27     // Insertando al indice
28     listaEjemplo.insertarNodoIndice(15, 2);
29     listaEjemplo.imprimirLista();
30     listaEjemplo.insertarNodoIndice(3, 4);
31     listaEjemplo.imprimirLista();
32 }
```

¿Cómo estamos contando?

```

1  public void insertarNodoIndice(int dato, int idx){
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < idx && nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     nodoIndice.siguiente = nodoRecorre.siguiente;
11     nodoRecorre.siguiente = nodoIndice;
12 }

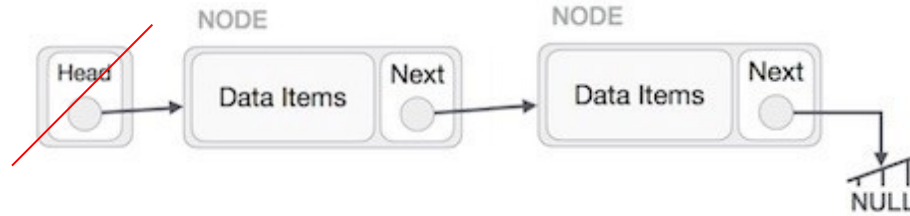
```

```

1  public void insertarNodoIndice(int dato, int idx) {
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < (idx-1) && nodoRecorre.siguiente != null) { // Empezando desde la posición 1
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     if (cont==(idx-1)){
11         nodoIndice.siguiente = nodoRecorre.siguiente; // Primero conectamos el Indice con lo que queda hasta la cola
12         nodoRecorre.siguiente = nodoIndice;          // Despues conectamos lo del inicio con el Indice
13     }
14 }
15 }

```

Eliminando elementos **al inicio** de una Lista (**DIY**)



```

1  public void eliminarNodoInicio() {
2      Nodo inicio = cabeza;
3      cabeza = inicio.siguiete;
4      inicio.siguiete = null; //romper el enlace
5      tamano--;
6  }

```

Opción 2:

```

1  public void eliminarNodoInicio() {
2      cabeza = cabeza.siguiete;
3      tamano--;
4  }

```

Conociendo el tamaño de la lista? (DIY)

```
1  private int tamano;  
2  
3  public int getTamano(){  
4      return tamano;  
5  }  
//NOTA: Se debe agregar los correspondientes tamano++ y tamano--
```

Verificar si la lista está vacía? (DIY)

```
1  public boolean estaVacia(){  
2      if(cabeza == null){  
3          return true;  
4      }  
5      else{  
6          return false;  
7      }  
8  }
```

```

1  public class Lista {
2
3      private Nodo cabeza;
4      Private int tamano;
5
6      public Lista(){
7          this.cabeza = null;
8          this.tamano = 0;
9      }
10 }

```

```

1  public void insertarNodoFinal(int dato){
2
3      Nodo nodoFin = new Nodo(dato);
4      Nodo nodoRecorre = cabeza;
5
6      while (nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8      }
9      nodoRecorre.siguiente = nodoFin;
10     tamano++;
11 }

```

```

1  public void insertarNodoIndice(int dato, int idx) {
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < (idx-1) && nodoRecorre.siguiente != null) {
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     if (cont==(idx-1)){
11         nodoIndice.siguiente = nodoRecorre.siguiente; // Primero conectamos el Indice con lo que queda hasta la cola
12         nodoRecorre.siguiente = nodoIndice;           // Despues conectamos lo del inicio con el Indice
13         tamano++;
14     }
15 }

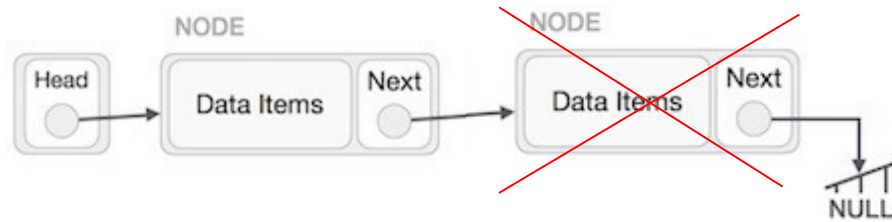
```

```

1  public void insertarNodoInicio(int dato){
2
3      Nodo nodoIni = new Nodo(dato);
4      nodoIni.siguiente = cabeza;
5      cabeza = nodoIni;
6      Tamano++;
7  }

```

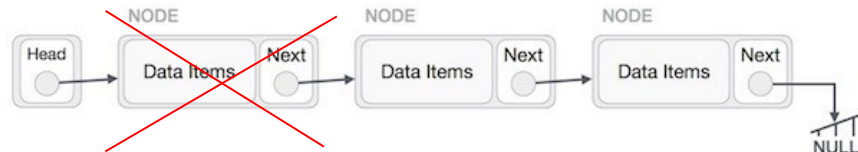

Eliminando elementos **al final** de una Lista (**DIY**)



```

1  public void eliminarNodoFinal() {
2      if (cabeza.siguiete == null) { //Si solo hay uno (corresponde a la cabeza)
3          cabeza = null; // Entonces hacer la lista vacía
4      } else {
5          Nodo nodoRecorre = cabeza;
6
7          while (nodoRecorre.siguiete.siguiete != null) {
8              nodoRecorre = nodoRecorre.siguiete;
9          }
10         nodoRecorre.siguiete = null;
11     }
12     tamano--;
13 }
    
```

Eliminando elementos **al índice indicado** de una Lista (**DIY**)



```

1  public void eliminarNodoIndice(int idx) {
2      Nodo nodoRecorre = cabeza;
3
4      int cont = 0;
5      while (cont < (idx - 1) && nodoRecorre.siguiete.siguiete != null) {
6          nodoRecorre = nodoRecorre.siguiete;
7          cont++;
8      }
9
10     Nodo aux = nodoRecorre.siguiete;           //Nodo a eliminar
11     nodoRecorre.siguiete = aux.siguiete;
12     aux.siguiete = null;                       //romper el enlace
13
14     tamano--;
15 }

```

```

1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Eliminando al inicio");
8      // Eliminando al principio
9      listaEjemplo.eliminarNodoInicio();
10     listaEjemplo.imprimirLista();
11     System.out.println("Inicio: Tamaño de la lista es: "+listaEjemplo.getTamano());
12
13     listaEjemplo.eliminarNodoInicio();
14     listaEjemplo.imprimirLista();
15     System.out.println("Inicio: Tamaño de la lista es: "+listaEjemplo.getTamano());
16
17     System.out.println();
18     System.out.println("Eliminando al final");
19     listaEjemplo.eliminarNodoFinal();
20     listaEjemplo.imprimirLista();
21     System.out.println("Tamaño de la lista es: "+listaEjemplo.getTamano());
22
23     System.out.println();
24     System.out.println("Eliminando en índice");
25     listaEjemplo.eliminarNodoIndice(2);
26     listaEjemplo.imprimirLista();
27     System.out.println("Idx: Tamaño de la lista es: "+listaEjemplo.getTamano());
28
29     listaEjemplo.eliminarNodoIndice(1);
30     listaEjemplo.imprimirLista();
31     System.out.println("Idx: Tamaño de la lista es: "+listaEjemplo.getTamano());
32 }
33 }
    
```

Buscando elementos de una Lista (**DIY**)

```
1  public int buscarNodo(int idx){
2      Nodo nodoRecorre = cabeza;
3
4      int cont = 0;
5      while (cont < idx && nodoRecorre.siguiente != null){
6          nodoRecorre = nodoRecorre.siguiente;
7          cont++;
8      }
9      return nodoRecorre.dato;
10 }
11
12
13
```

Probando...



```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Buscando Nodo");
8      // Buscar nodo al indice idx
9      int idx = 2;
10     int valDato = listaEjemplo.buscarNodo(idx);
11     System.out.println("Valor del nodo (" + idx + ") = " + valDato);
12     System.out.println("Tamaño de la lista es: " + listaEjemplo.getTamano());
13 }
14 }
```

Buscando elementos de una Lista (**DIY**)

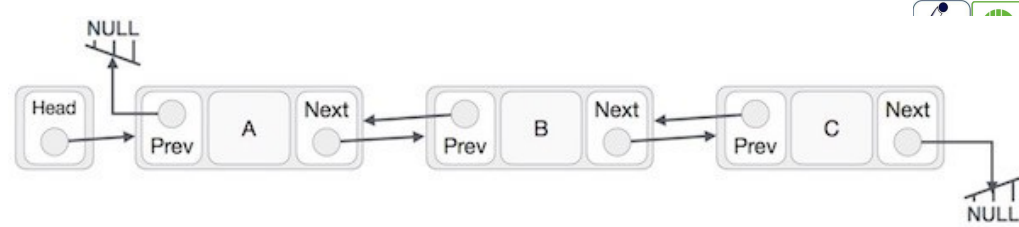
Ejercicio: ¿Cómo hacemos para buscar un “valor” específico y no una posición?

```
1  public int buscarNodoValor(int val) {  
2      Nodo nodoRecorre = cabeza;  
3  
4      int cont = 0;  
5      while (nodoRecorre.dato!=val && nodoRecorre.siguiente != null) {  
6          nodoRecorre = nodoRecorre.siguiente;  
7          cont++;  
8      }  
9      return cont;  
10 }
```

Ejercicio: ¿Cómo hacemos para buscar un “valor” específico y no una posición?

```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Buscando Nodo con valor");
8      // Buscar nodo con valor val
9      int val = 20;
10     int pos = listaEjemplo.buscarNodoValor(val);
11     System.out.println("Posicion del nodo con valor (" + val + ") = " + pos);
12     System.out.println("Tamaño de la lista es: " + listaEjemplo.getTamano());
13 }
14 }
```

1.2 Lista Enlazada Doble

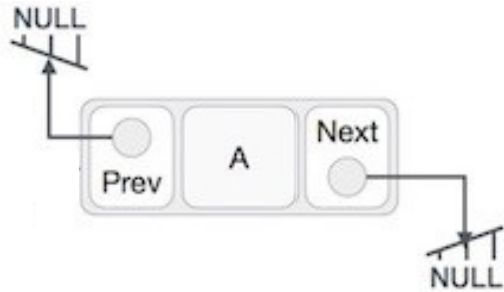


Estructura: Nodo (con **datos** y 1 **puntero** al **siguiente** y 1 puntero al **anterior**)

Operaciones permitidas:

- **Creación**
- **Insertar** elementos (al inicio, al final, al índice indicado)
- **Recorrer** la lista para imprimirla
- Verificar si está **vacía**
- **Buscar** elementos
- **Eliminar** elementos (al inicio, al final, al índice indicado)

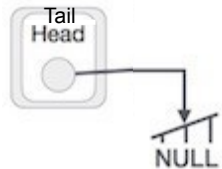
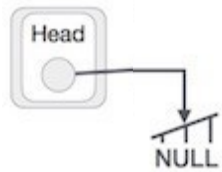
Construyendo un Nodo para Lista doble (DIY)



```

1  public class Nodo2 {
2
3      public int dato;
4      public Nodo2 siguiente = null;
5      public Nodo2 anterior= null;
6
7      public Nodo2(int dato){
8          this.dato = dato;
9      }
10
11 }
    
```

Construyendo una Lista (**DIY**)



```
1  public class Lista2 {  
2  
3      private Nodo2 cabeza, cola;  
4  
5      public Lista2(){  
6          this.cabeza = null;  
7          this.col = null;  
8      }  
9  }
```

Conociendo el tamaño de la Lista? (DIY)

```
1  private int tamano;  
2  
3  public int getTamano(){  
4      return tamano;  
5  }  
//NOTA: Se debe agregar los correspondientes tamano++ y tamano--
```

Verificar si la Lista está vacía? (DIY)

```
1  public boolean estaVacia(){  
2      if(cabeza == null){  
3          return true;  
4      }  
5      else{  
6          return false;  
7      }  
8  }
```

Imprimiendo elementos de una Lista (**DIY**)

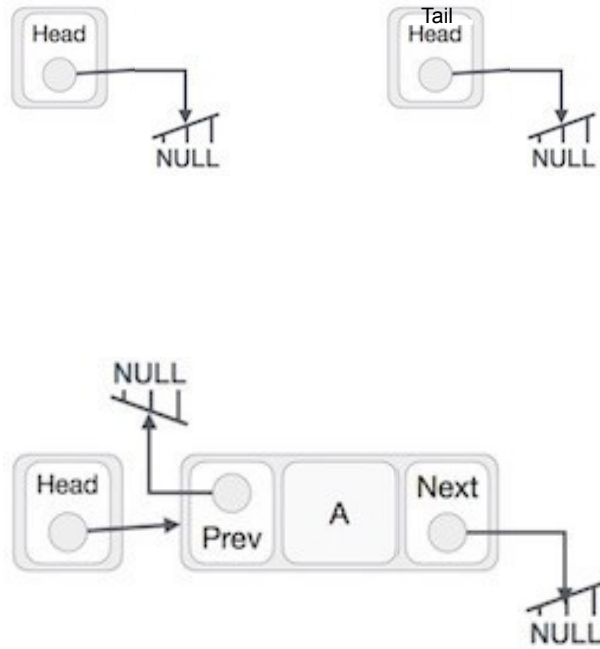
De inicio a fin

```
1  public void imprimirListaIniAFin() {  
2      Nodo2 nodoRecorre = cabeza;  
3  
4      System.out.print("CABEZA -> ");  
5      while (nodoRecorre != null) {  
6          System.out.print(nodoRecorre.dato + " -> ");  
7          nodoRecorre = nodoRecorre.siguiente;  
8      }  
9      System.out.println("FINAL");  
10 }
```

De fin a inicio

```
1  public void imprimirListaFinAIni() {  
2      Nodo2 nodoRecorre = cola;  
3  
4      System.out.print("FINAL -> ");  
5      while (nodoRecorre != null) {  
6          System.out.print(nodoRecorre.dato + " -> ");  
7          nodoRecorre = nodoRecorre.anterior;  
8      }  
9      System.out.println("INICIO");  
10 }
```

Insertando elementos **al inicio** de una Lista (**DIY**)

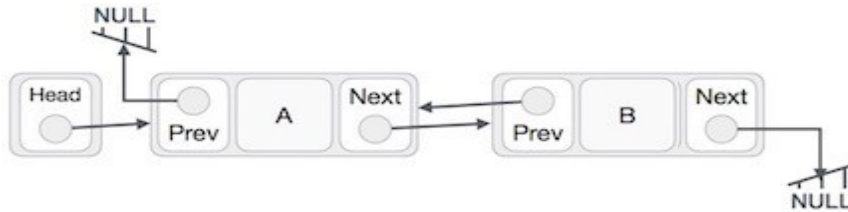


```

1  public void insertarNodoInicio(int dato) {
2      Nodo2 nuevo = new Nodo2(dato);
3
4      if (estaVacia()) {
5          cola = nuevo;
6      } else {
7          cabeza.anterior = nuevo;
8      }
9
10     nuevo.siguiente = cabeza;
11     nuevo.anterior = null;
12
13     cabeza = nuevo;
14     tamano++;
15 }

```

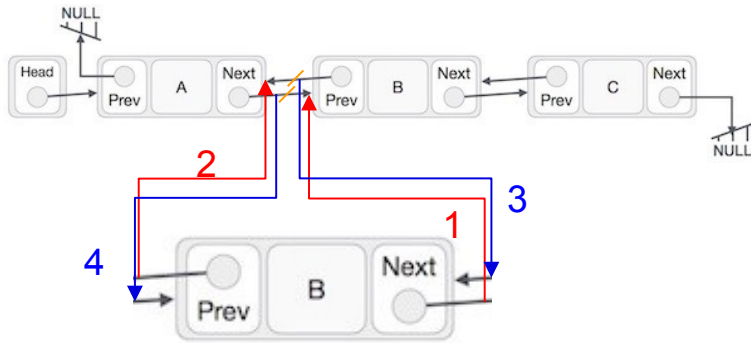
Insertando elementos al final de una Lista (DIY)



```

1  public void insertarNodoFinal(int dato) {
2      Nodo2 nuevo = new Nodo2(dato);
3
4      if (estaVacia() == true) { // Si está vacía la lista
5          cabeza = nuevo;
6      } else {
7          cola.siguiente = nuevo;
8          nuevo.anterior = cola;
9      }
10
11     cola = nuevo;
12     tamano++;
13 }
  
```

Insertando elementos al índice indicado de una Lista (DIY)



```

1  public void insertarNodoPosicion(int dato, int pos) {
2      Nodo2 nuevo = new Nodo2(dato);
3      Nodo2 recorre;
4
5      if (estaVacia() == true && pos != 0) { // Si está vacía la lista
6          System.out.println("Posicion no existe");
7      } else if (estaVacia() == true && pos == 0) {
8          cabeza = nuevo;
9          cola = nuevo;
10     } else {
11         recorre = cabeza;
12         int cont = 0;
13
14         while (cont < pos-1 && recorre.siguiente != null) {
15             recorre = recorre.siguiente;
16             cont++;
17         }
18
19         if (cont == pos-1) {
20             nuevo.siguiente = recorre.siguiente;
21             nuevo.anterior = recorre;
22
23             recorre.siguiente.anterior = nuevo;
24             recorre.siguiente = nuevo;
25         } else {
26             System.out.println("Posicion no existe");
27         }
28     }
29     tamaño++;
30 }
31

```

Probando...



```
1 public class Listas_Enlazadas_Dobles {
2     public static void main(String[] args) {
3         // TODO code application logic here
4         Lista2 listaEjemplo = new Lista2();
5         System.out.println("Nueva lista");
6         listaEjemplo.imprimirListaIniAFin();
7         listaEjemplo.imprimirListaFinAIni();
8
9         System.out.println("");
10        System.out.println("Insertando al final");
11        listaEjemplo.insertarNodoFinal(10);
12        listaEjemplo.imprimirListaIniAFin();
13        //listaEjemplo.imprimirListaFinAIni();
14
15        System.out.println("");
16        System.out.println("Insertando al inicio");
17        listaEjemplo.insertarNodoInicio(5);
18        listaEjemplo.imprimirListaIniAFin();
19
20        System.out.println("");
21        System.out.println("Insertando al final");
22        listaEjemplo.insertarNodoFinal(20);
23        listaEjemplo.imprimirListaIniAFin();
24
25        System.out.println("");
26        System.out.println("Insertando al final");
27        listaEjemplo.insertarNodoFinal(30);
28        listaEjemplo.imprimirListaIniAFin();
29
30        System.out.println("");
31        System.out.println("Insertando Nodo en posicion 2");
32        listaEjemplo.insertarNodoPosicion(1, 2);
33        listaEjemplo.imprimirListaIniAFin();
34    }
35 }
```


Eliminando elementos **al inicio/final** de una Lista (**DIY**)

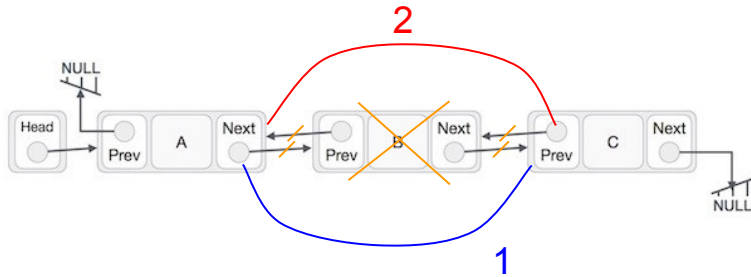
Eliminar al INICIO

```
1  public void eliminarNodoInicio() {
2      if (estaVacia()) {
3          System.out.println("Nada por eliminar");
4      } else if (cabeza == cola) {
5          cola = null;
6          cabeza = null;
7      } else {
8          cabeza = cabeza.siguiente;
9          cabeza.anterior = null;
10         tamano--;
11     }
12 }
```

Eliminar al FINAL

```
1  public void eliminarNodoFinal() {
2      if (estaVacia()) {
3          System.out.println("Nada por eliminar");
4      } else if (cabeza == cola) {
5          cola = null;
6          cabeza = null;
7      } else {
8          cola = cola.anterior;
9          cola.siguiente = null;
10         tamano--;
11     }
12 }
```

Eliminando elementos al índice indicado de una Lista (DIY)



```

1  public void eliminarNodoPos(int pos) {
2      Nodo2 recorre;
3
4      if (estaVacia()) {
5          System.out.println("Nada por eliminar");
6      } else if (cabeza == cola) {
7          cola = null;
8          cabeza = null;
9      } else {
10         recorre = cabeza;
11         int cont = 0;
12
13         while (cont < pos && recorre.siguiente != null) {
14             recorre = recorre.siguiente;
15             cont++;
16         }
17
18         if (cont == pos) {
19             recorre.siguiente = recorre.siguiente.siguiente;
20             recorre.siguiente.anterior = recorre;
21             tamano--;
22         } else {
23             System.out.println("Posicion no existente");
24         }
25     }
26 }
    
```

Probando...



```
1  public class Listas_Enlazadas_Dobles {
2      public static void main(String[] args) {
3          // TODO code application logic here
4          Lista2 listaEjemplo = new Lista2();
5          System.out.println("Nueva lista");
6          listaEjemplo.imprimirListaIniAFin();
7          listaEjemplo.imprimirListaFinAIni();
8
9          System.out.println("");
10         System.out.println("Eliminando nodo del inicio");
11         listaEjemplo.eliminarNodoInicio();
12         listaEjemplo.imprimirListaIniAFin();
13
14         System.out.println("");
15         System.out.println("Eliminando nodo en posicion 1");
16         listaEjemplo.eliminarNodoPos(1);
17         listaEjemplo.imprimirListaIniAFin();
18
19         System.out.println("");
20         System.out.println("Eliminando nodo del final");
21         listaEjemplo.eliminarNodoFinal();
22         listaEjemplo.imprimirListaIniAFin();
23
24         System.out.println("");
25         System.out.println("Eliminando nodo del inicio");
26         listaEjemplo.eliminarNodoInicio();
27         listaEjemplo.imprimirListaIniAFin();
28
29         System.out.println("");
30         System.out.println("Eliminando nodo del final");
31         listaEjemplo.eliminarNodoFinal();
32         listaEjemplo.imprimirListaIniAFin();
33     }
34 }
35 }
```

Buscando elementos de una Lista (DIY)

```
1 public int buscarNodo(int idx) {
2     Nodo2 nodoRecorre = cabeza;
3
4     int cont = 0;
5     while (cont < idx && nodoRecorre.siguiente != null) {
6         nodoRecorre = nodoRecorre.siguiente;
7         cont++;
8     }
9
10    if (cont == idx){
11        return nodoRecorre.dato;
12    }
13    else{
14        System.out.println("Posicion no existe");
15        return 0;
16    }
17 }
```

```
1 public int buscarValor(int val) {
2     Nodo2 nodoRecorre = cabeza;
3
4     int cont = 0;
5     while (nodoRecorre.dato!=val && nodoRecorre.siguiente != null) {
6         nodoRecorre = nodoRecorre.siguiente;
7         cont++;
8     }
9
10    if (nodoRecorre.dato==val){
11        return cont;
12    }
13    else{
14        System.out.println("Dato no existe");
15        return 0;
16    }
17 }
```

Ejercicio: ¿Cómo hacemos para buscar un “valor” específico y no una posición?

Probando...



```
1  public class Listas_Enlazadas_Dobles {
2      public static void main(String[] args) {
3          // TODO code application logic here
4          Lista2 listaEjemplo = new Lista2();
5          System.out.println("Nueva lista");
6          listaEjemplo.imprimirListaIniAFin();
7          listaEjemplo.imprimirListaFinAIni();
8
9          System.out.println("");
10         System.out.println("Buscando nodo en posicion 2");
11         int idx = 2;
12         int nodob = listaEjemplo.buscarNodo(idx);
13         System.out.println("Valor del nodo(" + idx + ") = " + nodob);
14         listaEjemplo.imprimirListaIniAFin();
15
16         System.out.println("");
17         System.out.println("Buscando dato 20");
18         int val = 20;
19         int pos = listaEjemplo.buscarValor(val);
20         System.out.println("Posicion del valor (" + val + ") = " + pos);
21         listaEjemplo.imprimirListaIniAFin();
22     }
23 }
```


1.2 Pilas (Last-In First-Out)

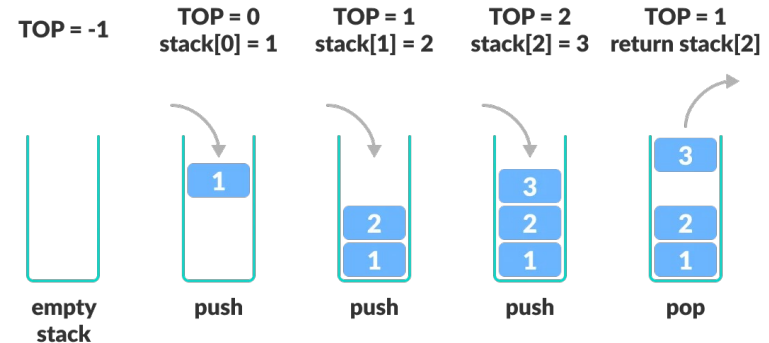
- Es una estructura de datos lineal que sigue el principio de último en entrar primero en salir (LIFO).
- Operaciones permitidas:
 - **push()**: agrega un elemento a la parte superior
 - **pop()**: elimina un elemento de la parte superior
 - **peek()**: obtiene el valor de la cima sin eliminarlo
 - **isEmpty()**: comprueba si la pila está vacía
 - **isFull()**: comprueba si la pila está llena



Aplicaciones:

- Análisis de sintaxis
- Comprobación de paréntesis en aritmética
 - $2 + 4 / 5 * (7 - 9)$
- Control de funciones o subrutinas activas

```
c[d]    // correct
a{b[c]}e // correct
a{b[c]d}e // not correct
```



Creando Pilas

```
1  public class Nodo {
2      public int dato;
3      public Nodo siguiente;
4
5      public Nodo(int dato){
6          this.dato = dato;
7          siguiente = null;
8      }
9  }
```

```
1  public class Pila {
2      private Nodo cima;
3      private int tamano;
4
5      public Pila(){
6          cima = null;
7          tamano = 0;
8      }
9  }
```

NOTA: Igual que una lista enlazada simple pero SOLO permite AGREGAR o ELIMINAR al INICIO; además, la **cabeza** ahora se llama **cima**

Operaciones: **push()** y **pop()**

```
1  public void push(int dato){
2      Nodo nuevo = new Nodo(dato);
3      nuevo.siguiente = cima;
4      cima = nuevo;
5      tamano++;
6      System.out.println("Se ingresó el elemento "+ dato+ " a la pila.");
7  }
```

```
1  public void insertarNodoInicio(int dato){
2
3      Nodo nodoIni = new Nodo(dato);
4      nodoIni.siguiente = cabeza;
5      cabeza = nodoIni;
        tamano++;
    }
```

```
1  public int pop() {
2      Nodo aux = cima;
3      cima = cima.siguiente;
4      tamano--;
5      System.out.println("Se sacó el elemento "+ aux.dato + " de la
6      pila.");
7      return aux.dato;
    }
```

```
1  public void eliminarNodoInicio() {
2      Nodo inicio = cabeza;
3      cabeza = inicio.siguiente;
4      inicio.siguiente = null; //romper el enlace
5      tamano--;
6  }
```

Operaciones: **peek()**, **isEmpty()**, **eliminarPila()**

```
1  public int peek(){
2      System.out.println("El elemento en la cima de la pila es "+ cima.dato);
3      return cima.dato;
4  }
```

```
1  public boolean isEmpty(){
2      if (cima == null){
3          System.out.println("La pila está vacía");
4          return true;
5      }
6      else{
7          System.out.println("La pila no está vacía");
8          return false;
9      }
10 }
```

```
1  public void eliminarPila(){
2      while(!isEmpty()){
3          pop();
4      }
5  }
```

Probando...



```
1  public class Pilas_Main {
2
3      /**
4       * @param args the command line arguments
5       */
6      public static void main(String[] args) {
7          // TODO code application logic here
8          Pila stack = new Pila();
9
10         stack.push(0);
11
12         stack.push(10);
13         stack.pop();
14         stack.isEmpty();
15         stack.peek();
16
17         stack.push(20);
18         stack.isEmpty();
19         stack.eliminarPila();
20
21     }
22
23 }
```

1.3 Colas (First-In First-Out)

- Es una estructura de datos lineal que sigue el principio de primero en entrar primero en salir (FIFO).
- Tipos: simples, circular, de prioridad, y doble
- Operaciones permitidas:
 - **enqueue()**: agrega un elemento al final de la cola
 - **dequeue()**: elimina un elemento del frente de la cola
 - **isEmpty()**: comprueba si la cola está vacía
 - **isFull()**: comprueba si la cola está llena
 - **peek()**: obtiene el valor del frente de la cola sin eliminarlo



Aplicaciones:

- Programación de tareas en un PC (imprimir, abrir programas, copiar archivos, etc.)
- Call centers (pedir citas, etc.)
- Listas de reproducción (música, videos youtube, etc)
- Envío de mensajes por Whatsapp y no se tiene conexión a internet



Creando Colas

```
1  public class Nodo {  
2      public int dato;  
3      public Nodo siguiente;  
4  
5      public Nodo(int dato){  
6          this.dato = dato;  
7          siguiente = null;  
8      }  
9  }
```

```
1  public class Cola {  
2      private Nodo inicio;  
3      private Nodo fin;  
4      private int tamano;  
5  
6      public Cola(){  
7          inicio = fin = null;  
8          tamano = 0;  
9      }  
}
```

NOTA: Igual que una lista enlazada simple pero SOLO permite AGREGAR al FINAL y ELIMINAR al INICIO

Operaciones: enqueue() y dequeue()

```
1  public void enqueue(int dato){
2      Nodo nuevo = new Nodo(dato);
3
4      if (isEmpty()){
5          inicio = nuevo;
6      }
7      else{
8          fin.siguiete = nuevo;
9      }
10
11     fin = nuevo;
12     tamano++;
13 }
```

```
1  public void insertarNodoFinal(int dato){
2      Nodo nuevo = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      while (nodoRecorre.siguiete != null){
6          nodoRecorre = nodoRecorre.siguiete;
7      }
8
9      nodoRecorre.siguiete = nuevo;
10     tamano++;
11 }
```

```
1  public int dequeue(){
2      Nodo aux = inicio;
3      inicio = inicio.siguiete;
4      aux.siguiete = null;
5      tamano--;
6      return aux.dato;
7  }
```

```
1  public void eliminarNodoInicio() {
2      Nodo inicio = cabeza;
3      cabeza = inicio.siguiete;
4      inicio.siguiete = null; //romper el enlace
5      tamano--;
6  }
```

Operaciones: **peek()**, **isEmpty()**, **eliminarCola()**

```
1  public int peek(){  
2      System.out.println("El primero en la cola es "+ inicio.dato);  
3      return inicio.dato;  
4  }
```

```
1  public boolean isEmpty(){  
2      if (inicio == null){  
3          System.out.println("La cola está vacía");  
4          return true;  
5      }  
6      else{  
7          System.out.println("La cola no está vacía");  
8          return false;  
9      }  
10 }
```

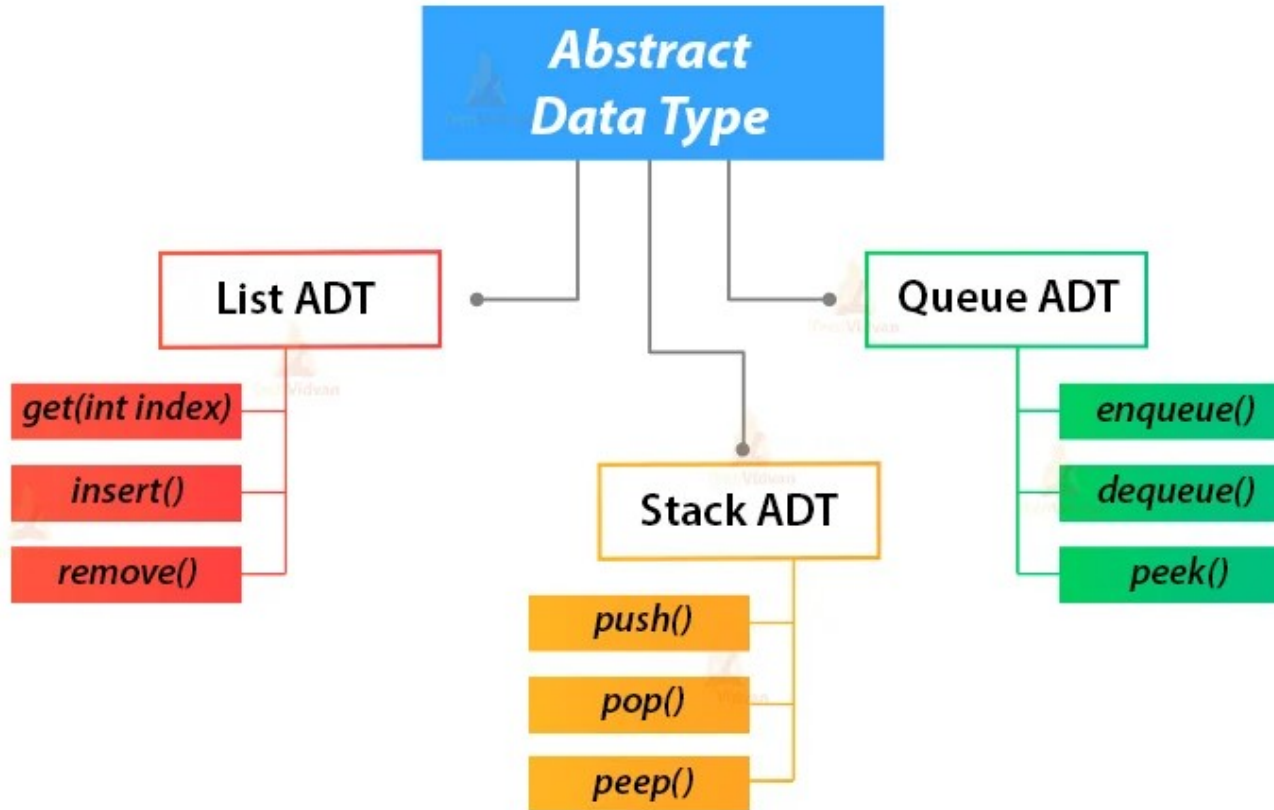
```
1  public void eliminarCola(){  
2      while(!isEmpty()){  
3          dequeue();  
4      }  
5  }
```

Probando...

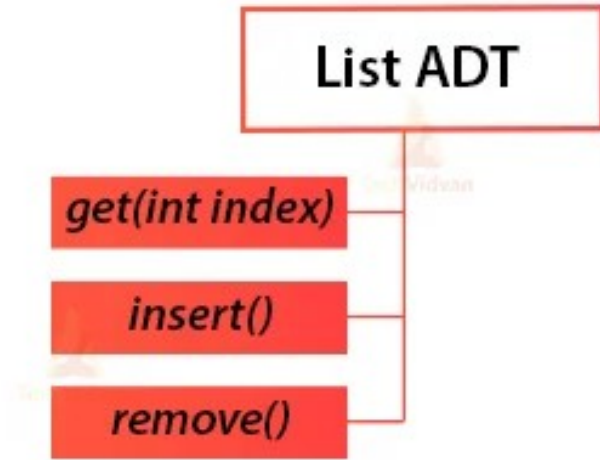


```
1  public class Colas_Main {
2
3      /**
4       * @param args the command line arguments
5       */
6      public static void main(String[] args) {
7          // TODO code application logic here
8
9          Cola colaejemplo = new Cola();
10
11          colaejemplo.enqueue(0);
12
13          colaejemplo.enqueue(10);
14          colaejemplo.dequeue();
15          colaejemplo.isEmpty();
16          colaejemplo.peek();
17
18          colaejemplo.enqueue(20);
19          colaejemplo.isEmpty();
20          colaejemplo.eliminarCola();
21
22          colaejemplo.isEmpty();
23      }
24
25 }
```


Various Java Abstract Data Types



Clase LinkedList de Java

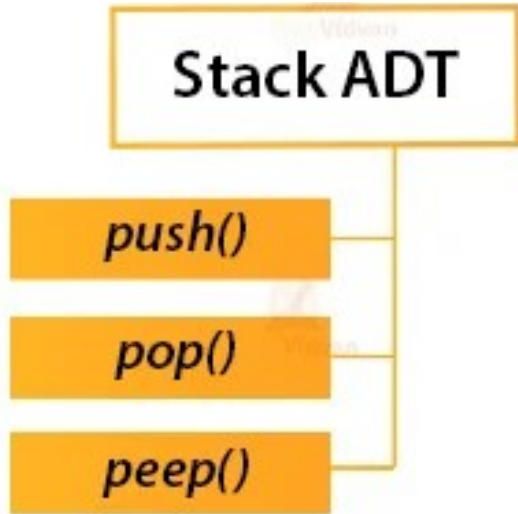


```
import java.util.LinkedList;
```

```

1  public static void main(String[] args) {
2      LinkedList lista = new LinkedList();
3      lista.add(10);
4      lista.add(30);
5      lista.add(20);
6      lista.add(50);
7      lista.add(1);
8      lista.add(8);
9      lista.add(1, 400);
10
11     int tamaño = lista.size();
12     int i=0;
13     System.out.print("INICIO");
14     while(i<tamaño){
15         System.out.print(" -> "+lista.get(i));
16         i++;
17     }
18     System.out.println(" -> NULL");
19
20     lista.removeFirst();
21     lista.removeLast();
22     lista.remove(2);
23 }
  
```

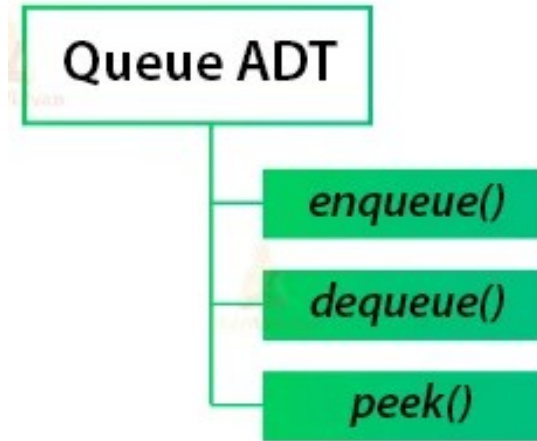
Clase Stack de Java



```
1  public static void main(String[] args){
2      Stack pila = new Stack();
3      pila.add(1);
4      pila.add(2);
5      pila.add(3);
6      pila.add(4);
7      pila.push(5);
8
9      System.out.println("La cima de la pila es "+pila.peek());
10     System.out.println("Sacando un elemento de la pila " + pila.pop());
11     System.out.println("Tamano de la pila es "+pila.size());
12
13     System.out.println("La cima de la pila es "+pila.peek());
14     System.out.println("Sacando un elemento de la pila " + pila.pop());
15
16     System.out.println("Tamano de la pila es "+pila.size());
17 }
```

```
import java.util.Stack;
```

Clase Queue de Java



```
1  public static void main(String[] args) {
2      Queue<Integer> cola = new LinkedList<Integer>();
3
4      cola.add(1);
5      cola.add(2);
6      cola.add(3);
7      cola.add(4);
8      cola.add(5);
9
10     System.out.println("Tamano de la cola "+cola.size());
11     System.out.println("El primero en la cola es "+cola.peek());
12     System.out.println("Eliminando el primero de la cola "+cola.poll());
13     System.out.println("Tamano de la cola "+cola.size());
14     System.out.println("El primero en la cola es "+cola.peek());
15 }
```

```
import java.util.Queue;
```

2. Estructuras de datos NO lineales

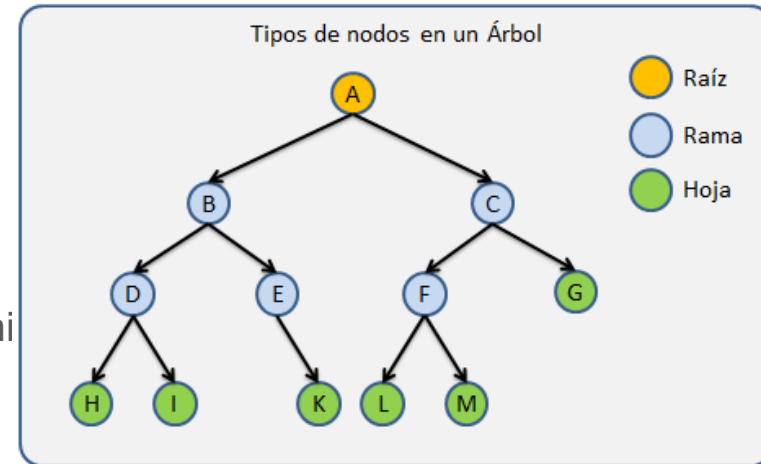
Árboles

- Un árbol es una estructura de datos jerárquica, no lineal, y dinámica que consta de **nodos** conectados por **aristas**.
- Existen 3 tipos de nodos: **Raíz, rama, hoja**.
- Tipos de árboles: **Binarios** y **multi-camino**
- Un árbol no debe tener ciclos
- Operaciones permitidas:
 - Búsqueda, inserción, eliminación



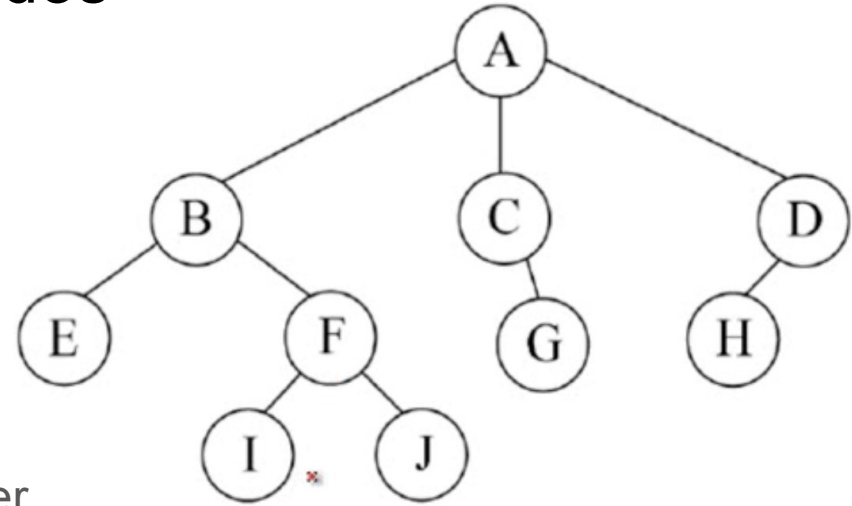
Aplicaciones:

- Los árboles son un método eficiente para búsquedas grandes y complejas
- Casi todos los sistemas operativos almacenan sus archivos en árboles (carpetas/subcarpetas)
- Indexación en bases de datos



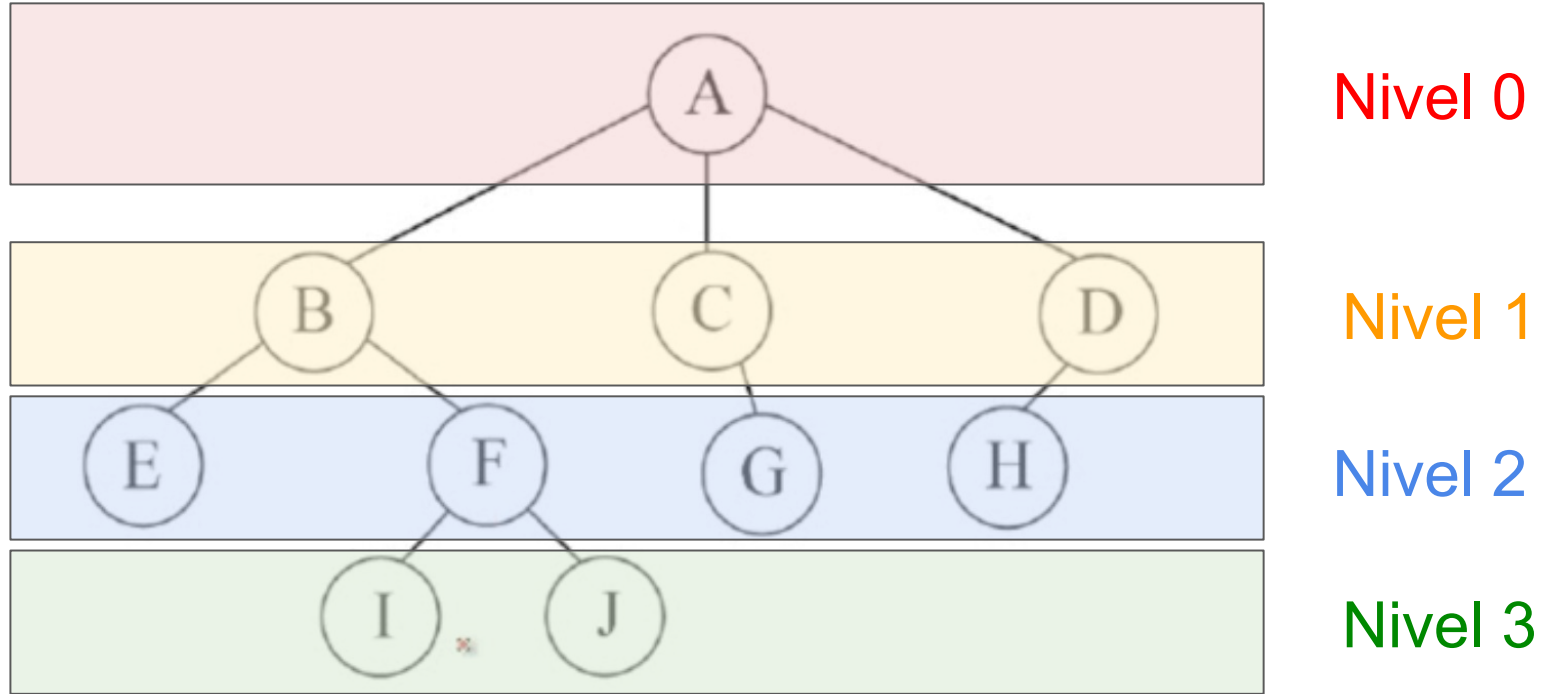
Árboles: Clasificación de los nodos

- Raíz o Padre:
 - Si tiene sucesores
- Rama o Hijo:
 - Descendientes del padre. Puede tener hermanos.
- Hoja:
 - Nodo hijo que no tiene sucesores



Padres? Hijos?
Hermanos? Hojas?

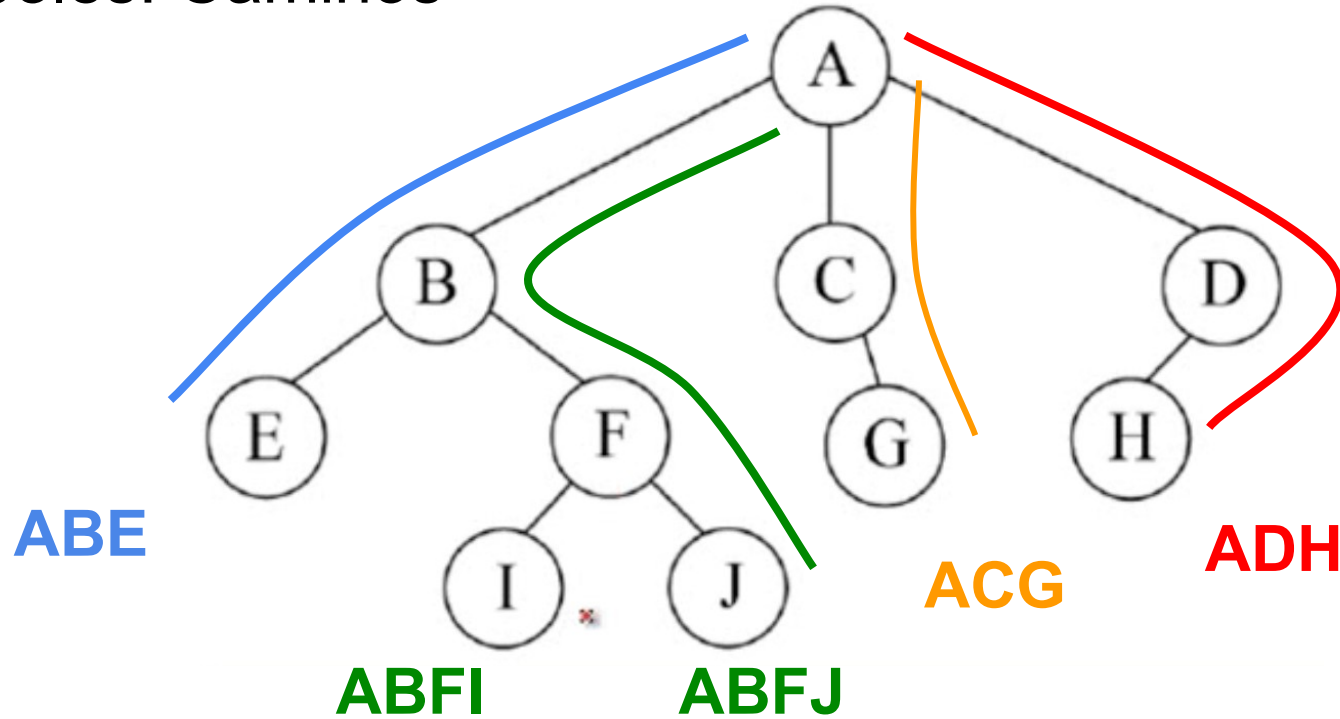
Árboles: Nivel de los nodos y altura de un árbol



Nivel = Distancia hasta el nodo raíz

Altura = Mayor nivel + 1

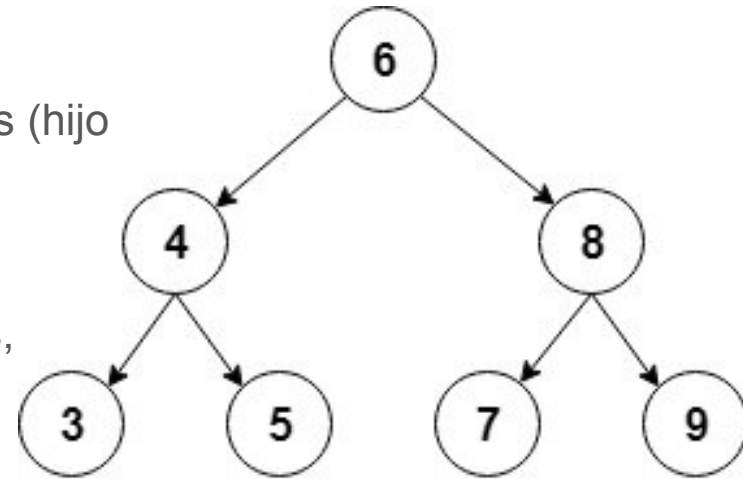
Árboles: Caminos



Camino: Secuencia de nodos desde la raíz hasta el nodo destino

Árboles Binarios

- Cuyos nodos no pueden tener más de dos sub-árboles
- Cada nodo puede tener 0, 1 o máximo 2 nodos hijos (hijo izquierdo o hijo derecho)
- Es una estructura recursiva (un árbol de subárboles, cada hijo es un árbol binario)
- **Árbol degenerado** es un árbol que solo tiene hijos en un solo sentido (se comporta como **lista enlazada**)

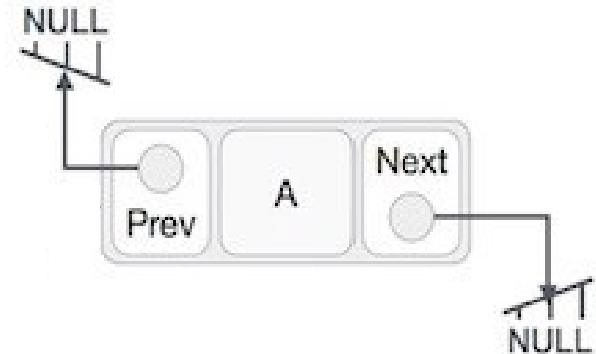
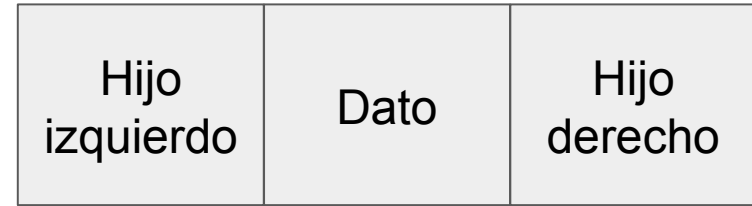


Árboles Binarios: Estructura de datos

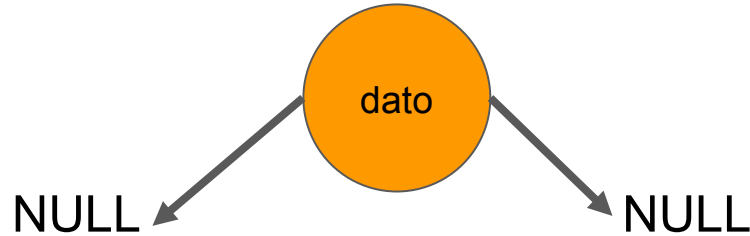
Operaciones básicas

- Creación
- Insertar nodo
- Saber si está vacío
- Recorrer árbol
- Buscar nodo
- Borrar nodo
- Obtener la raíz
- Determinar su altura (mayor nivel + 1)
- Determinar número de elementos (peso)

Nodo

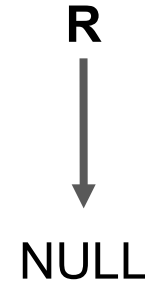


Árboles Binarios: Creación



```

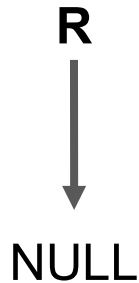
1  public class Nodo {
2      public int dato;
3      public Nodo hijoIzquierdo;
4      public Nodo hijoDerecho;
5
6      public Nodo(int dato){
7          this.dato = dato;
8          this.hijoIzquierdo = null;
9          this.hijoDerecho = null;
10     }
11 }
  
```



```

1  public class ArbolBinario {
2      Nodo raiz;
3
4      public ArbolBinario(){
5          this.raiz = null;
6      }
7  }
  
```

Árboles Binarios: **Verificar si está vacío**



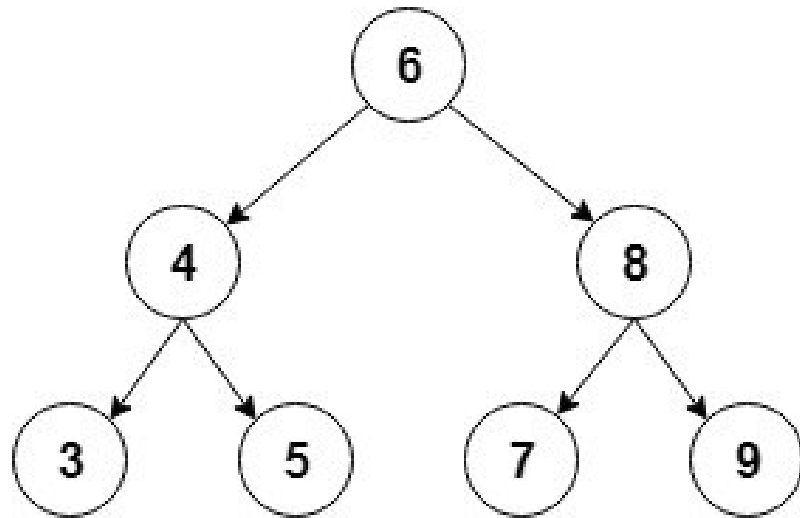
```
1  public boolean estaVacio(){  
2      if(raiz == null){  
3          return true;  
4      }  
5      else{  
6          return false;  
7      }  
8  }
```

Árboles Binarios: **Insertar** Nodo

- Se insertan de **manera ordenada**

Insertar: 6, 8, 4, 7, 9, 5, 3

- Se empieza por la raíz y se pregunta si es mayor o menor
- Si es mayor o **igual** se avanza por el nodo de la derecha; si es menor se avanza por el nodo de la izquierda
- Se inserta si y sólo si el puntero es NULL



Insertar (Método 1)



Insertar (Recursivo)

```
1  public void insertarNodo(int dato){
2      Nodo nuevo = new Nodo(dato);
3
4      if (raiz == null){
5          raiz = nuevo;
6      }
7      else{
8          Nodo recorre = raiz;
9          Nodo padre;
10         while(true){
11             padre = recorre;
12             if (dato < recorre.dato){
13                 recorre = recorre.hijoIzquierdo;
14                 if (recorre == null){
15                     padre.hijoIzquierdo = nuevo;
16                     return;
17                 }
18             }
19             else{
20                 recorre = recorre.hijoDerecho;
21                 if (recorre == null){
22                     padre.hijoDerecho = nuevo;
23                     return;
24                 }
25             }
26         }
27     }
28 }
```

```
1  public void insertarNodoRec(int dato){
2      raiz = insertarNodoRec(raiz, dato);
3  }
4
5  public Nodo insertarNodoRec(Nodo padre, int dato){
6      if (padre == null){
7          return new Nodo(dato);
8      }
9      else{
10         if (dato < padre.dato){
11             padre.hijoIzquierdo = insertarNodoRec(padre.hijoIzquierdo, dato);
12         }
13         else if (dato >= padre.dato){
14             padre.hijoDerecho = insertarNodoRec(padre.hijoDerecho, dato);
15         }
16         return padre;
17     }
18 }
```

Probando...

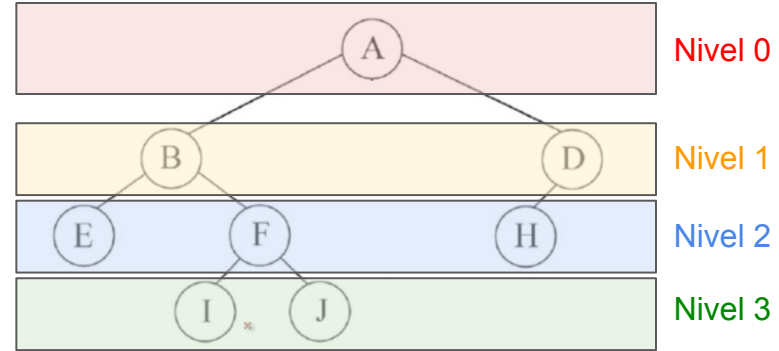


```
1  public static void main(String[] args) {  
2      //TODO code application logic here  
3      ArbolBinario ab = new ArbolBinario();  
4  
5      ab.insertarNodo(40);  
6      ab.insertarNodo(20);  
7      ab.insertarNodo(10);  
8      ab.insertarNodo(30);  
9      ab.insertarNodo(60);  
10     ab.insertarNodo(50);  
11     ab.insertarNodo(70);  
12     ab.insertarNodo(5);  
13     ab.insertarNodo(13);  
14     ab.insertarNodoRec(55);  
15 }
```


Árboles Binarios: Tipos de recorrido

- **En anchura:**

Procesar todos los nodos del mismo nivel y después pasar al siguiente nivel. **(ABDEFHIJ)**

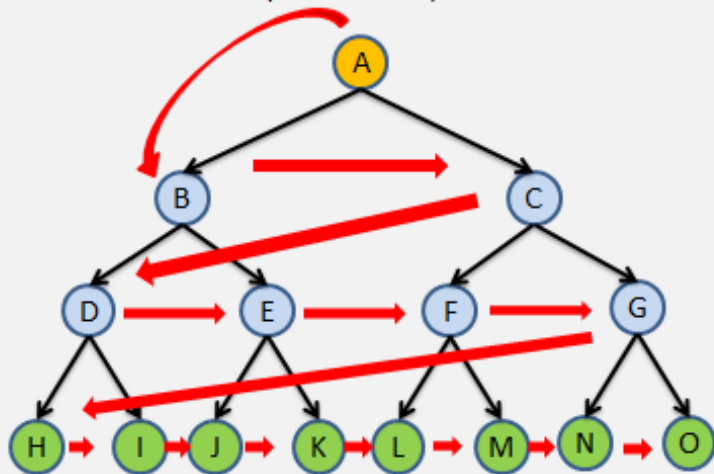


- **En profundidad**

- Pre-Orden: Raíz, sub-árbol izquierdo y sub-árbol derecho **(ABEFIJDH)**
- In-Orden: Sub-árbol izquierdo, raíz y sub-árbol derecho **(EBIFJAHD)**
- Post-Orden: Sub-árbol izquierdo, sub-árbol derecho y raíz **(EIJFBHDA)**

Árboles Binarios: Tipos de recorrido: **En anchura**

Búsqueda en amplitud.



Procedimiento:

1. Se agrega la Raíz a la **cola** de nodos por visitar
2. Mientras que la cola no esté vacía, se saca el primer elemento de la cola y vamos al paso 3. Pero si la cola está vacía entonces terminamos.
3. Se agregan todos los hijos del nodo a la cola de nodos pendientes por procesar.
4. Regresamos al paso 2.

```

1  public void recorrerAnchura(){
2      if (estaVacio()){
3          System.out.println("El arbol esta vacio");
4      }
5      else{
6          Queue<Nodo> nodos = new LinkedList<>();
7          nodos.add(raiz);
8
9          while (!nodos.isEmpty()) {
10             Nodo nodo = nodos.poll();
11
12             System.out.print(" " + nodo.dato);
13
14             if (nodo.hijoIzquierdo != null) {
15                 nodos.add(nodo.hijoIzquierdo);
16             }
17
18             if (nodo.hijoDerecho != null) {
19                 nodos.add(nodo.hijoDerecho);
20             }
21         }
22     }
23 }
    
```

Probando...



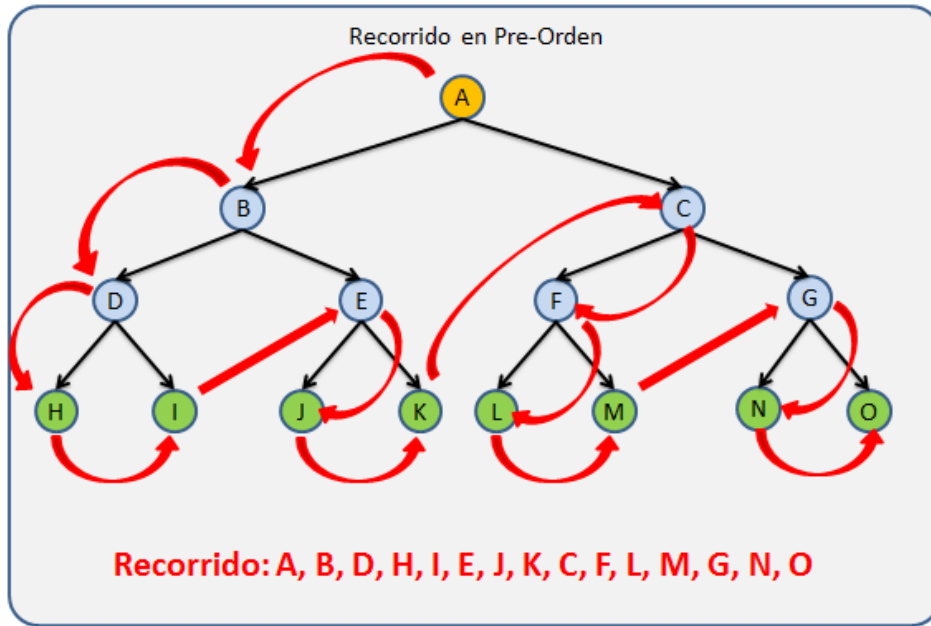
```
1  public static void main(String[] args) {  
2      //TODO code application logic here  
3      ArbolBinario ab2 = new ArbolBinario();  
4      ab2.insertarNodoRecursivo(10);  
5      ab2.insertarNodoRecursivo(5);  
6      ab2.insertarNodoRecursivo(4);  
7      ab2.insertarNodoRecursivo(13);  
8      System.out.println("\n Recorriendo en anchura");  
9      ab2.recorrerAnchura();  
10 }
```

Recorriendo en anchura

10 -> 5 -> 13 -> 4 ->

Árboles Binarios: Tipos de recorrido: **Pre-Orden (R,I,D)**

- **Pre-Orden:** Raíz, sub-árbol izquierdo y sub-árbol derecho



```

1  public void recorrerPreOrden(Nodo recorre){
2      if (recorre != null){
3          System.out.println(recorre.dato);
4          recorrerPreOrden(recorre.hijoIzquierdo);
5          recorrerPreOrden(recorre.hijoDerecho);
6      }
7  }
    
```

Probando...



```
1  public static void main(String[] args) {
2      //TODO code application logic here
3      ArbolBinario ab2 = new ArbolBinario();
4      ab2.insertarNodoRecursoivo(10);
5      ab2.insertarNodoRecursoivo(5);
6      ab2.insertarNodoRecursoivo(4);
7      ab2.insertarNodoRecursoivo(13);
8      System.out.println("\n Recorriendo en anchura");
9      ab2.recorrerAnchura();
10
11     System.out.println("\n Recorriendo en Pre-Orden");
12     ab2.recorrerPreOrden(ab2.raiz);
13 }
```

Recorriendo en anchura

10 -> 5 -> 13 -> 4 ->

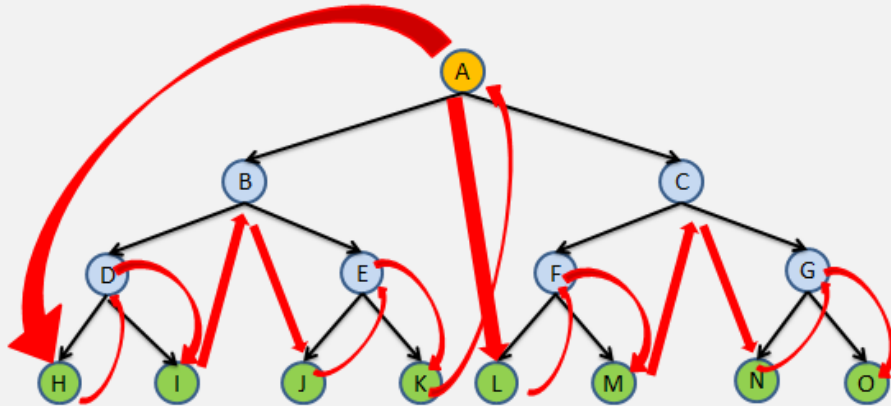
Recorriendo en Pre-Orden

10 -> 5 -> 4 -> 13 ->

Árboles Binarios: Tipos de recorrido: **In-Orden (I,R,D)**

- **In-Orden:** Sub-árbol izquierdo, raíz y sub-árbol derecho

Recorrido en In-Orden



Recorrido: H, D, I, B, J, E, K, A, L, F, M, C, N, G, O

```

1  public void recorrerInOrden(Nodo recorre){
2      if (recorre != null){
3          recorrerInOrden(recorre.hijoIzquierdo);
4          System.out.println(recorre.dato);
5          recorrerInOrden(recorre.hijoDerecho);
6      }
7  }
    
```

Probando...



```
1  public static void main(String[] args) {
2      //TODO code application logic here
3      ArbolBinario ab2 = new ArbolBinario();
4      ab2.insertarNodoRecursivo(10);
5      ab2.insertarNodoRecursivo(5);
6      ab2.insertarNodoRecursivo(4);
7      ab2.insertarNodoRecursivo(13);
8      System.out.println("\n Recorriendo en anchura");
9      ab2.recorrerAnchura();
10
11     System.out.println("\n Recorriendo en Pre-Orden");
12     ab2.recorrerPreOrden(ab2.raiz);
13
14     System.out.println("\n Recorriendo en In-Orden");
15     ab2.recorrerInOrden(ab2.raiz);
16 }
```

Recorriendo en anchura

10 -> 5 -> 13 -> 4 ->

Recorriendo en Pre-Orden

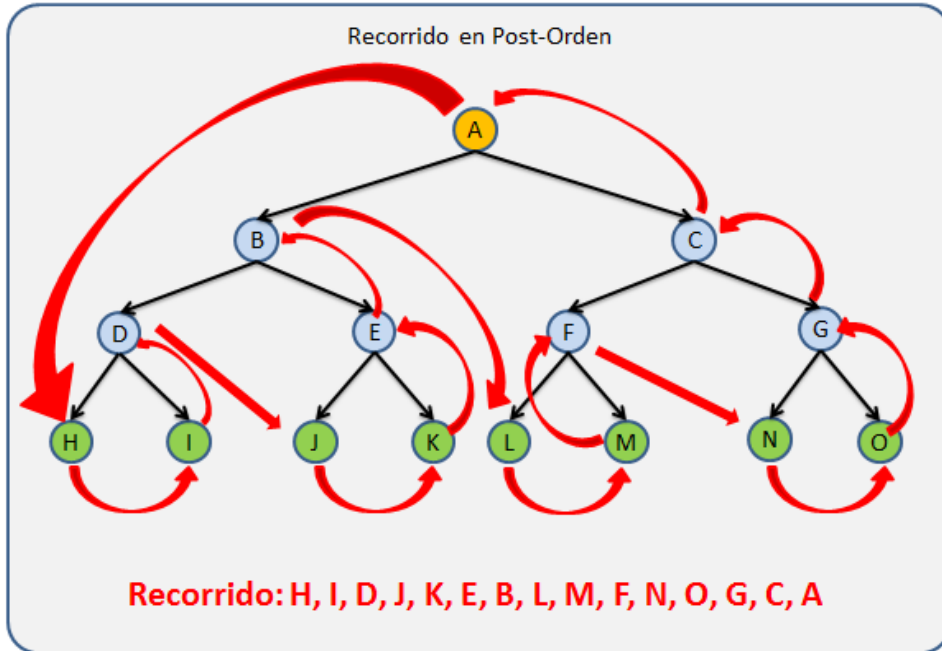
10 -> 5 -> 4 -> 13 ->

Recorriendo en In-Orden

4 -> 5 -> 10 -> 13 ->

Árboles Binarios: Tipos de recorrido: **Post-Orden (I,D,R)**

- **Post-Orden:** Sub-árbol izquierdo, sub-árbol derecho y raíz



```

1  public void recorrerPostOrden(Nodo recorre){
2      if (recorre != null){
3          recorrerPostOrden(recorre.hijoIzquierdo);
4          recorrerPostOrden(recorre.hijoDerecho);
5          System.out.println(recorre.dato);
6      }
7  }
    
```


Probando...

```
1 public static void main(String[] args) {
2     //TODO code application logic here
3     ArbolBinario ab2 = new ArbolBinario();
4     ab2.insertarNodoRecursivo(10);
5     ab2.insertarNodoRecursivo(5);
6     ab2.insertarNodoRecursivo(4);
7     ab2.insertarNodoRecursivo(13);
8     System.out.println("\n Recorriendo en anchura");
9     ab2.recorrerAnchura();
10
11     System.out.println("\n Recorriendo en Pre-Orden");
12     ab2.recorrerPreOrden(ab2.raiz);
13
14     System.out.println("\n Recorriendo en In-Orden");
15     ab2.recorrerInOrden(ab2.raiz);
16
17     System.out.println("\n Recorriendo en Post-Orden");
18     ab2.recorrerPostOrden(ab2.raiz);
19 }
```

Recorriendo en anchura

10 -> 5 -> 13 -> 4 ->

Recorriendo en Pre-Orden

10 -> 5 -> 4 -> 13 ->

Recorriendo en In-Orden

4 -> 5 -> 10 -> 13 ->

Recorriendo en Post-Orden

4 -> 5 -> 13 -> 10 ->



Árboles Binarios: **Búsqueda**

1. Crear nodo **recorrer** que empiece en raíz
2. Mientras el dato buscado sea diferente al dato del nodo recorrer, hacer:
 - a. Si el dato buscado es menor que el dato del nodo recorrer, entonces recorra el árbol por la izquierda
 - b. Si no, recorra el árbol por la derecha
 - c. Verificar si el nodo recorrer es nulo (alcanzamos el fin y no encontramos el dato)
3. Retornar el nodo **recorrer**

```
1  public Nodo buscarNodo(int dato){
2      Nodo recorrer = raiz;
3
4      while(recorre.dato != dato){
5          if (dato < recorrer.dato){
6              recorrer = recorrer.hijoIzquierdo;
7          }
8          else{
9              recorrer = recorrer.hijoDerecho;
10         }
11         if (recorre == null){
12             return null;
13         }
14     }
15
16     return recorrer;
17 }
```

Probando...

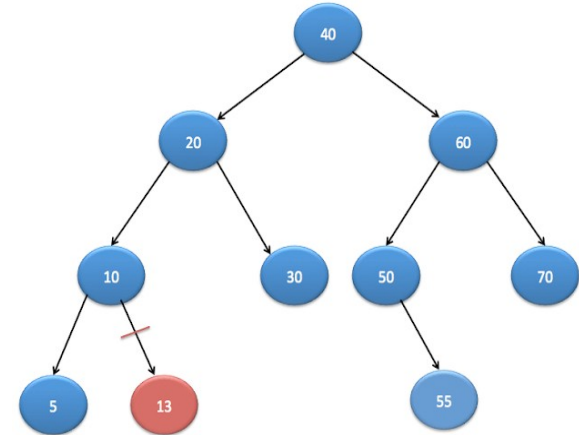


```
1  public static void main(String[] args) {
2      //TODO code application logic here
3      ArbolBinario ab = new ArbolBinario();
4
5      ab.insertarNodo(40);
6      ab.insertarNodo(20);
7      ab.insertarNodo(10);
8      ab.insertarNodo(30);
9      ab.insertarNodo(60);
10     ab.insertarNodo(50);
11     ab.insertarNodo(70);
12     ab.insertarNodo(5);
13     ab.insertarNodo(13);
14     ab.insertarNodoRecursivo(55);
15     int datoBuscar = 40;
16     Nodo aux = ab.buscarNodo(datoBuscar);
17     if (aux == null){
18         System.out.println("No se encontró el nodo con el dato: "+datoBuscar);
19     }
20     else{
21         System.out.println("Si se encontró el nodo con dato: "+aux.dato);
22     }
23 }
```

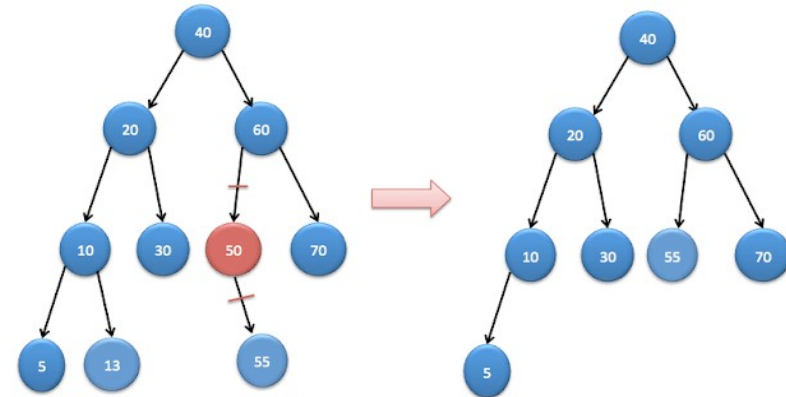
Árboles Binarios: **Eliminar un nodo**

Hay tres casos al eliminar un nodo:

1. Si el nodo **no** tiene hijos
 - a. Buscamos el nodo y lo hacemos NULL
2. Si el nodo tiene **1 hijo**
 - a. El hijo pasa a la posición del padre
3. Si el nodo tiene **2 hijos (complicated)**
 - a. Necesitamos conectar el padre del nodo al nodo más a la izquierda (mínimo) del subárbol derecho o al nodo más a la derecha (máximo) del subárbol izquierdo.

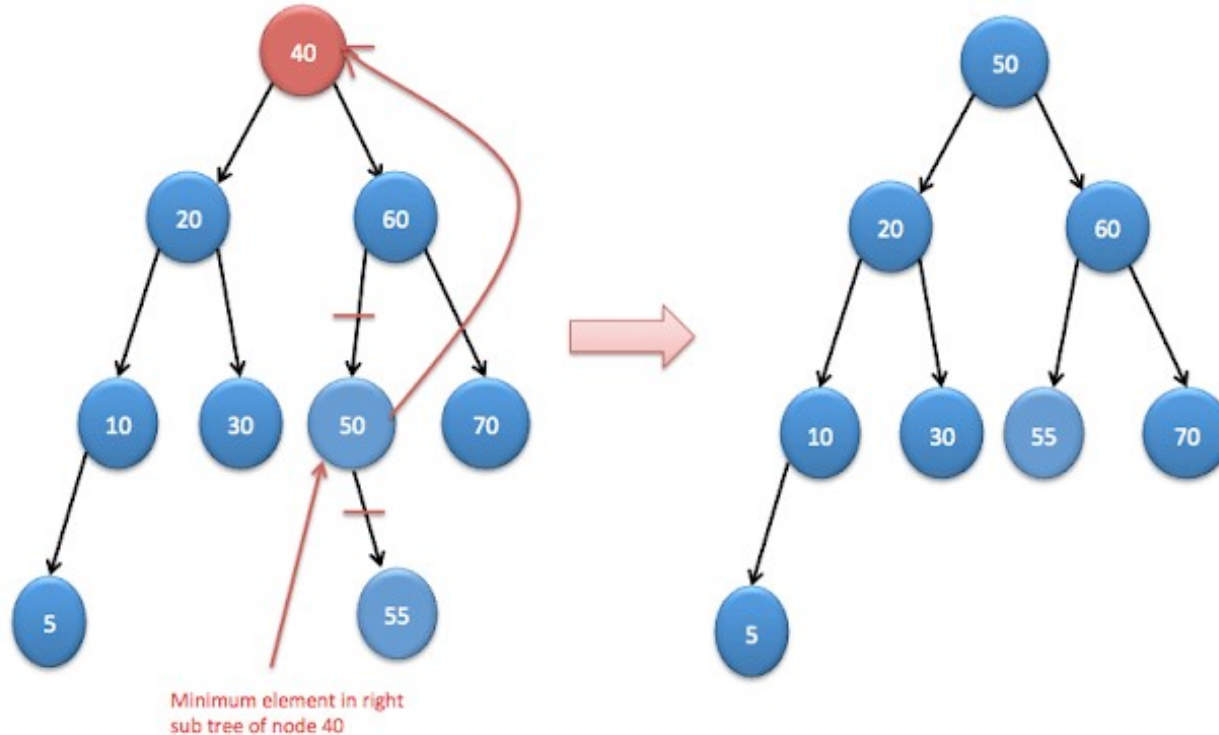


Deleting node 50 which have one child



Árboles Binarios: **Eliminar un nodo con 2 hijos**

Deleting node 40 which have two children



Árboles Binarios: Eliminar un nodo con 2 hijos

```

1  public void eliminarNodo(int dato){
2      eliminarNodoRec(raiz, dato);
3  }

```

```

1  public Nodo minimoElemento(Nodo recorre) {
2      if (recorre.hijoIzquierdo == null)
3          return recorre;
4      else
5          return
6      minimoElemento(recorre.hijoIzquierdo);
7  }

```

```

1  public Nodo eliminarNodoRec(Nodo recorre, int dato) {
2      if (recorre == null) //No tiene mas hijos
3          return null;
4      if (dato < recorre.dato) { //Buscando nodo a eliminar
5          recorre.hijoIzquierdo = eliminarNodoRec(recorre.hijoIzquierdo,
6          dato);
7      } else if (dato > recorre.dato) {
8          recorre.hijoDerecho = eliminarNodoRec(recorre.hijoDerecho, dato);
9      } else {
10         // Si el nodo a borrar tiene 2 hijos
11         if (recorre.hijoIzquierdo != null && recorre.hijoDerecho != null) {
12             Nodo temp = recorre;
13             // busque el menor nodo de la derecha, reemplace recorre y
14             borrelo
15             Nodo minNodoDerecha = minimoElemento(temp.hijoDerecho);
16             recorre.dato = minNodoDerecha.dato;
17             recorre.hijoDerecho = eliminarNodoRec(recorre.hijoDerecho,
18             minNodoDerecha.dato);
19         }
20         // Si el nodo a borrar tiene 1 hijo (hijo izquierda); asiguelo
21         else if (recorre.hijoIzquierdo != null) {
22             recorre = recorre.hijoIzquierdo;
23         }
24         // Si el nodo a borrar tiene 1 hijo (hijo derecha); asiguelo
25         else if (recorre.hijoDerecho != null) {
26             recorre = recorre.hijoDerecho;
27         }
28         // Si el nodo a borrar no tiene hijos (hoja); borrelo
29         else
30             recorre = null;
31     }
32     return recorre;
33 }

```

Probando...



```
1  public static void main(String[] args) {
2      //TODO code application logic here
3      ArbolBinario ab = new ArbolBinario();
4
5      ab.insertarNodo(40);
6      ab.insertarNodo(20);
7      ab.insertarNodo(10);
8      ab.insertarNodo(30);
9      ab.insertarNodo(60);
10     ab.insertarNodo(50);
11     ab.insertarNodo(70);
12     ab.insertarNodo(5);
13     ab.insertarNodo(13);
14     ab.insertarNodoRecursivo(55);
15
16     System.out.println("\n Deleting node 40 which have two children:");
17     ab.eliminarNodo(40);
18 }
```

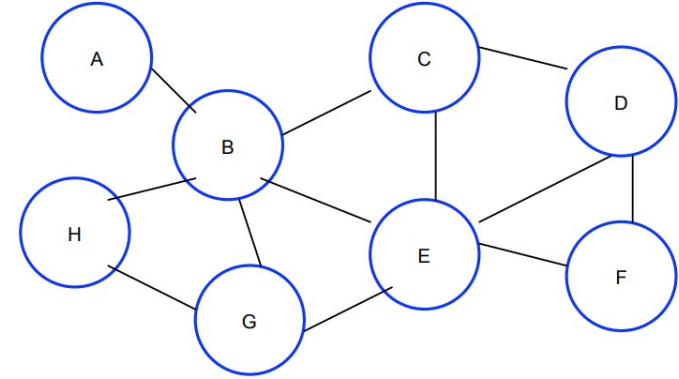

Grafos

$$G = (V, A)$$

$$V = \{A, B, C, D, E, F, G, H\}$$

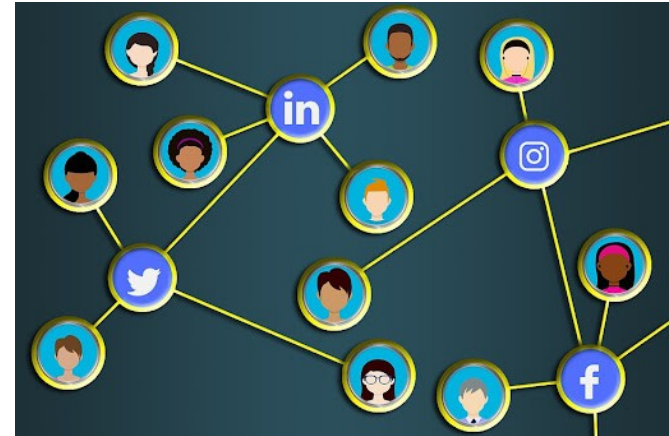
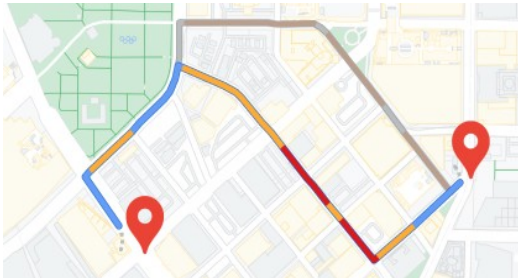
$$A = \{ \{A, B\}, \{B, C\}, \{B, E\}, \{B, G\}, \{B, H\}, \{C, D\}, \{C, E\}, \{D, F\}, \{D, E\}, \{E, F\}, \{E, G\}, \{G, H\} \}$$

- Un grafo, G , es un par, compuesto por dos conjuntos, uno de **vértices** (V) y uno de **aristas** (A).
 - A es un conjunto de pares de vértices, estos pares se conocen con el nombre de arcos o ejes del grafo.
- Se suele utilizar la notación $G = (V, A)$ para identificar un grafo.
- Tipos de grafos: Dirigidos, no dirigidos, etiquetados, ponderados



Aplicaciones:

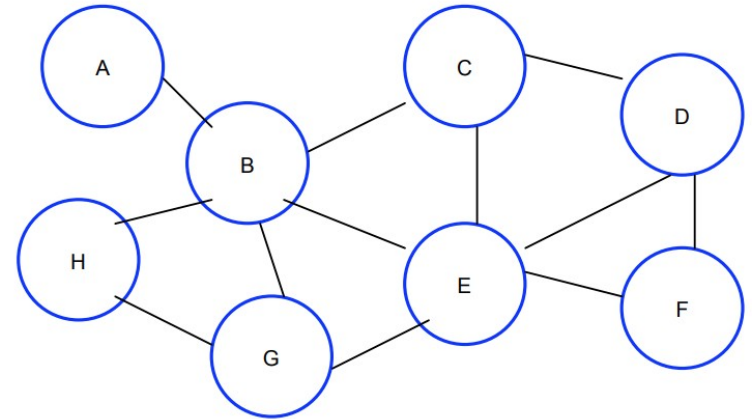
- Representación de redes y rutas en aplicaciones de comunicación, transporte y viajes.
- Interconexiones en redes sociales.



Grafos: Conceptos

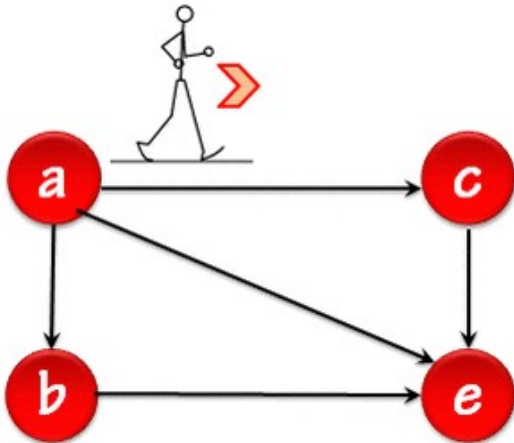


- Dirigido vs No dirigido
- Ponderado (con pesos) vs No-ponderado (sin pesos)
- Camino entre dos vértices
- Ciclo y bucle
- Grafo conexo y fuertemente conexo
- Árbol recubridor mínimo

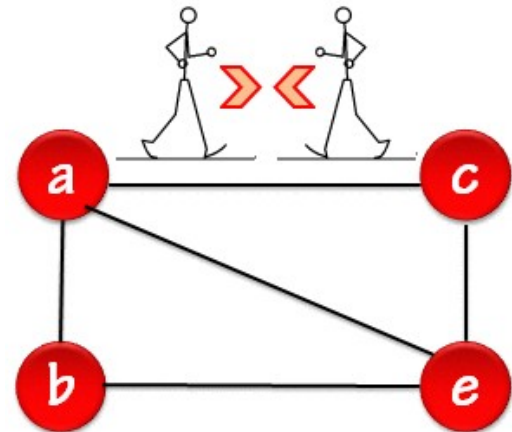


Grafos: Dirigido (digrafo) vs No dirigido

- Un **grafo dirigido** es aquel en el que todas sus aristas tienen sentido o dirección
- La relación sobre V **no es simétrica**
- Las aristas se representan como un **par ordenado (u,v)**
 - $V = \{a, b, c, e\}$
 - $E = \{(a, b), (a, c), (a, e), (b, e), (c, e)\}$

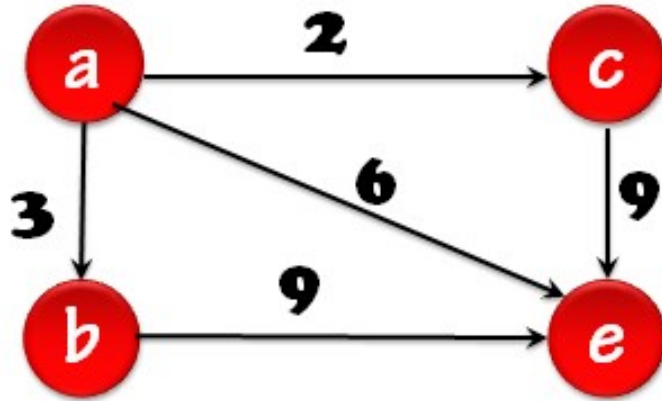


- Un grafo **no dirigido** es aquel en el que todas sus aristas son bidireccionales
- La relación sobre V **es simétrica**
- Las aristas se representan como **pares no ordenados $\{u,v\}$** , $u,v \in V$ y $u \neq v$
 - $V = \{a, b, c, e\}$
 - $E = \{\{a, b\}, \{a, c\}, \{a, e\}, \{b, e\}, \{c, e\}\}$



Grafos: Ponderado vs No ponderado

- Un **grafo ponderado**, o con pesos, es un grafo donde cada arista tiene asociado un valor o etiqueta, para representar el costo, peso, longitud, etc, de ir de un vértice al otro.
- En un grafo ponderado llamamos **peso de un camino** a la suma de los pesos de las aristas que lo forman.

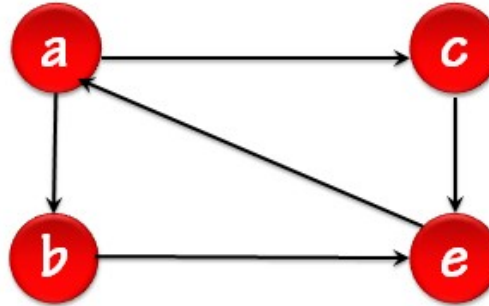
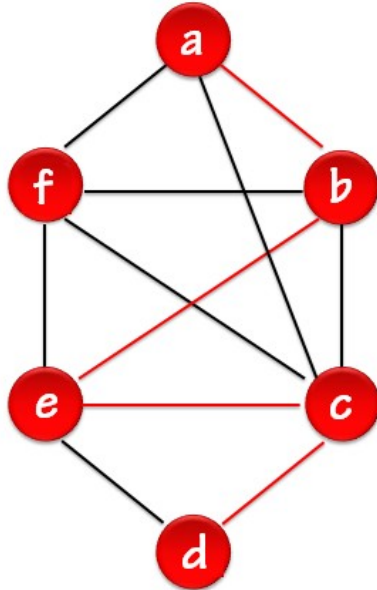


- En un grafo ponderado llamamos **camino más corto** entre dos vértices dados al camino de peso mínimo entre dichos vértices.
- En un grafo ponderado llamaremos **camino más largo** (o camino crítico) entre dos vértices al camino de peso máximo entre dichos vértices.

Grafos: Camino, ciclo y bucle

Camino $a \rightarrow d$:

- a, b, e, c, d : $\{\{a, b\}, \{b, e\}, \{e, c\}, \{c, d\}\}$, long:4
- a, b, e, f, b, e, c, d : long:7

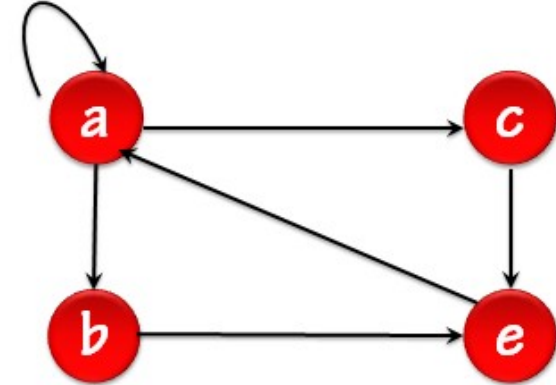


Ciclo: Camino donde el origen del camino es igual a su destino $a \rightarrow a$:

- a, c, e, a : $\{(a, c), (c, e), (e, a)\}$ Long:3
- a, b, e, a : $\{(a, b), (b, e), (e, a)\}$ Long:3

Bucle: arista que conecta a un vértice consigo mismo $a \rightarrow a$:

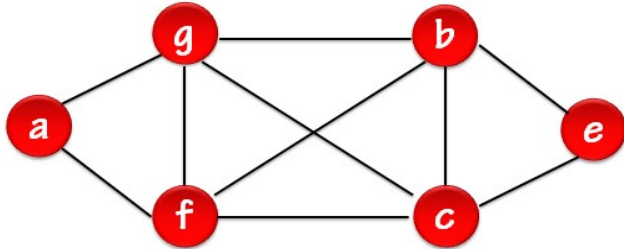
- $a, a = \{(a, a)\}$ Long: 1



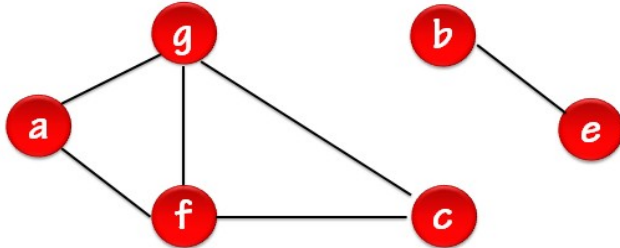
Grafos: Conexo y fuertemente conexo

Un grafo **no dirigido** es **conexo** si existe un camino entre cada par de vértices.

Conexo

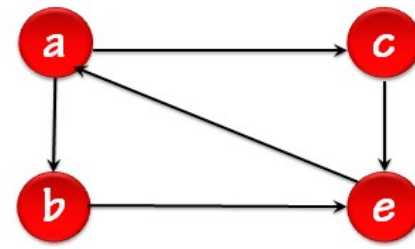


No Conexa

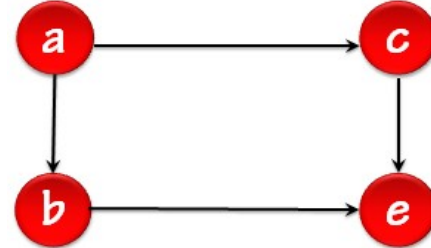


Un grafo **dirigido** es **fuertemente conexo** si existe un camino desde cualquier vértice a cualquier otro.

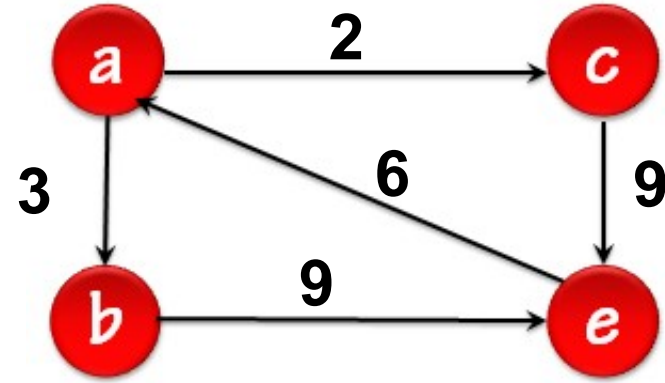
Fuertemente



Débilmente



Grafos: Representación mediante Matriz de adyacencia (**A**) y Matriz de pesos (**W**)



$$A = \begin{matrix} & \begin{matrix} a & b & c & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ e \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$W = \begin{matrix} & \begin{matrix} a & b & c & e \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ e \end{matrix} & \begin{bmatrix} 0 & 3 & 2 & \infty \\ \infty & 0 & \infty & 9 \\ \infty & \infty & 0 & 9 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

Grafos en Java: Con matriz de adyacencia

```

1  public class Grafo_Matriz_Adyacencia {
2      private int V; // numero de vertices
3      private int A; // numero de aristas
4      private int[][] matrizAdyacencia;
5
6      public Grafo_Matriz_Adyacencia(int nodos) {
7          this.V = nodos;
8          this.A = 0;
9          this.matrizAdyacencia = new int[nodos][nodos];
10     }
11 }

```

```

1  public void imprimirGrafo() {
2      for (int v = 0; v < V; v++) {
3          System.out.print("Fila " + v + ": ");
4          for (int w = 0; w < V; w++) {
5              System.out.print(matrizAdyacencia[v][w] + " ");
6          }
7          System.out.println("");
8      }
9  }

```

```

1  public void agregarArista(int u, int v) {
2      matrizAdyacencia[u][v] = 1;
3      matrizAdyacencia[v][u] = 1; // no-dirigido
4      A++;
5  }

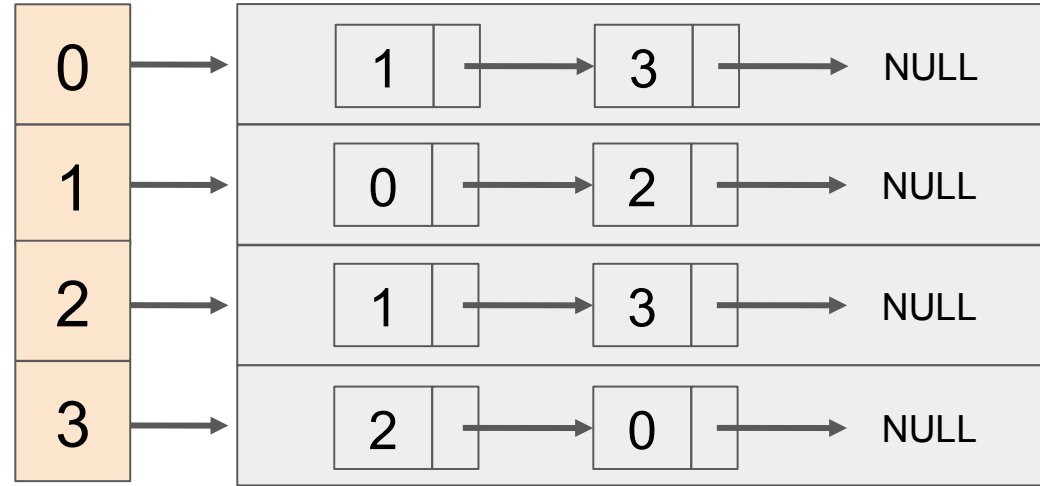
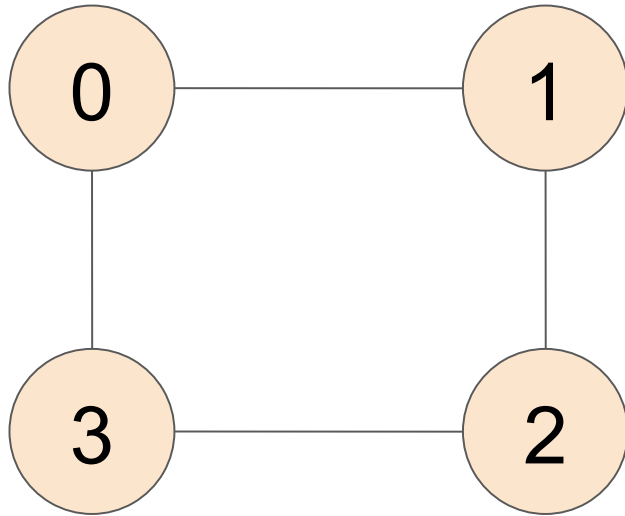
```

```

1  public static void main(String[] args) {
2      // TODO code application logic here
3      Grafo_Matriz_Adyacencia g = new
4      Grafo_Matriz_Adyacencia(4);
5      g.agregarArista(0, 1);
6      g.agregarArista(1, 2);
7      g.agregarArista(2, 3);
8      g.agregarArista(3, 0);
9      g.imprimirGrafo();
10 }

```


Grafos en Java: Representación con **listas de adyacencia**



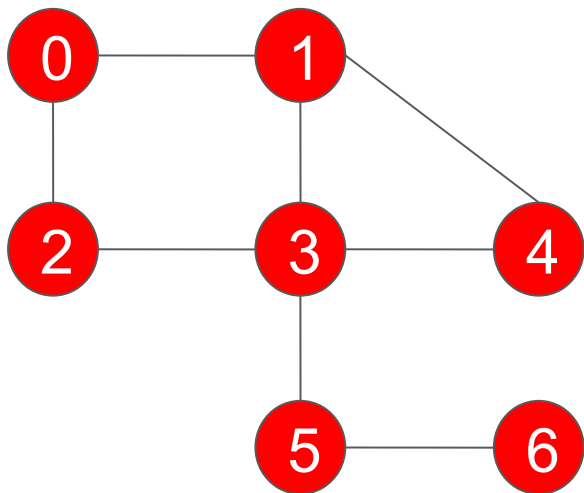
Grafos en Java: Con listas de adyacencia

```
1  public class Grafo_Listas_Adyacencia {
2
3      private LinkedList<Integer>[] adj;
4      private int V; // number of vertices
5      private int A; // number of aristas
6
7      public Grafo_Listas_Adyacencia(int nodos) {
8          this.V = nodos;
9          this.A = 0;
10         this.adj = new LinkedList[nodos];
11         for (int v = 0; v < V; v++) {
12             adj[v] = new LinkedList<>();
13         }
14     }
15 }
```

```
1  public void agregarArista(int u, int v) {
2      adj[u].add(v);
3      adj[v].add(u);
4      A++;
5  }
```

```
1  public void imprimirGrafo(){
2      for(int v = 0; v < V; v++) {
3          System.out.print("Row "+v+": ");
4          for(int w = 0; w < adj[v].size(); w++) {
5              System.out.print(adj[v].get(w) + " ");
6          }
7          System.out.println("");
8      }
9  }
```

Grafos: Recorrer en anchura (BFS) → Por niveles



1. Usa una cola (Queue = FIFO)
2. Agrega a la cola el primer nodo (cualquiera)
3. Remueve el primero de la cola (lo imprime) y agrega todos sus nodos adyacentes a la cola. Si no hay adyacentes sin visitar, no agrega nada y continúa.
4. Repite 3 para el siguiente nodo en la cola

Salida	Cola	Visita/agrega
	I 0 F	0
0	1 2	1, 2
1	2 3 4	3, 4
2	3 4	Nada
3	4 5	5
4	5	Nada
5	6	6
6		

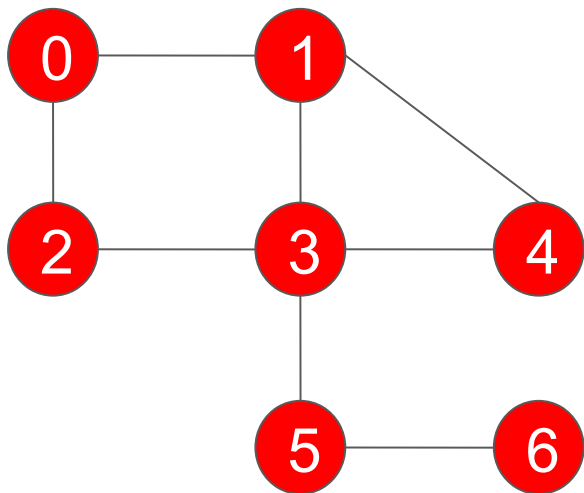
BFS=Breadth First Search

Grafos: Recorrer en anchura (BFS) en Java

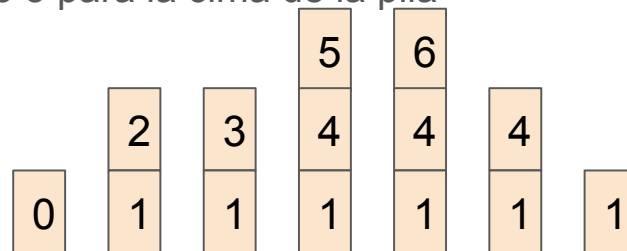
```
1  public void bfs(int s) {
2      boolean[] visited = new boolean[V];
3
4      Queue<Integer> q = new LinkedList<>();
5      visited[s] = true;
6      q.offer(s);
7
8      while (!q.isEmpty()) {
9          int u = q.poll();
10         System.out.print(u + " ");
11
12         for (int v : adj[u]) {
13             if (!visited[v]) {
14                 visited[v] = true;
15                 q.offer(v);
16             }
17         }
18     }
19 }
```

```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Grafo_Listas_Adyacencia g = new Grafo_Listas_Adyacencia(7);
4      g.agregarArista(0, 1);
5      g.agregarArista(0, 2);
6      g.agregarArista(2, 3);
7      g.agregarArista(1, 3);
8      g.agregarArista(1, 4);
9      g.agregarArista(3, 4);
10     g.agregarArista(3, 5);
11     g.agregarArista(5, 6);
12     g.imprimirGrafo();
13     System.out.println("BFS");
14     g.bfs(0);
15 }
```

Grafos: Recorrer en profundidad (DFS)



1. Usa una pila (Stack = LIFO)
2. Agrega a la pila el nodo inicial (cualquiera)
3. Elimina la cima, pero busca sus nodos “más cercanos” (no visitados) y los agrega a la pila. Si no hay más cercanos sin visitar, entonces continúa.
4. Repite el paso 3 para la cima de la pila



Salida:	0	2	3	5	6	4	1
Visita:	0	2	3	5	6	4	1
Agrega:	1,2	3	4,5	6			

Grafos: Recorrer en profundidad (DFS) en Java

```

1  public void dfs(int s) {
2      boolean[] visited = new boolean[V];
3      Stack<Integer> stack = new Stack<>();
4      stack.push(s);
5
6      while (!stack.isEmpty()) {
7          int u = stack.pop();
8          if (!visited[u]) {
9              visited[u] = true;
10             System.out.print(u + " ");
11
12             for (int v : adj[u]) {
13                 if (!visited[v]) {
14                     stack.push(v);
15                 }
16             }
17         }
18     }

```

```

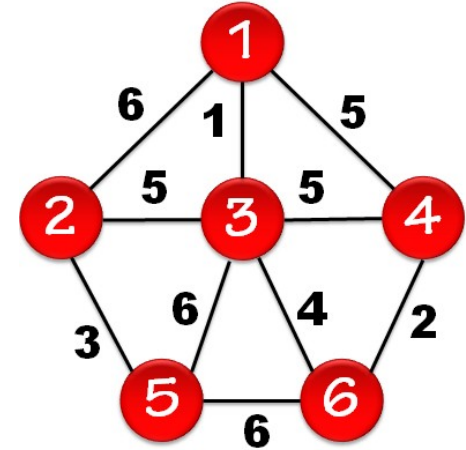
1  public static void main(String[] args) {
2      Grafo_Listas_Adyacencia g = new
3      Grafo_Listas_Adyacencia(7);
4      g.agregarArista(0, 1);
5      g.agregarArista(0, 2);
6      g.agregarArista(2, 3);
7      g.agregarArista(1, 3);
8      g.agregarArista(1, 4);
9      g.agregarArista(3, 4);
10     g.agregarArista(3, 5);
11     g.agregarArista(5, 6);
12     g.imprimirGrafo();
13     System.out.println("BFS");
14     g.bfs(0);
15     System.out.println("\nDFS");
16     g.dfs(0);
17 }

```

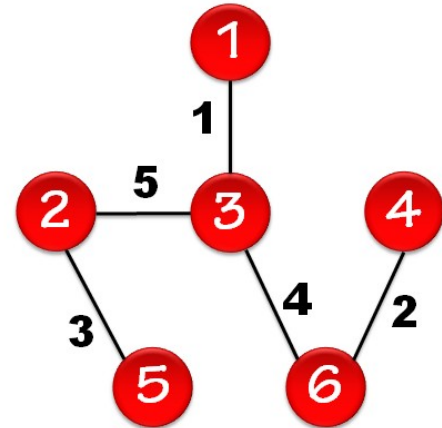
Grafos: **Árbol recubridor mínimo**

- El árbol recubridor mínimo es un árbol formado por las aristas de G que conectan todos los vértices V con un costo total mínimo
- No contiene ciclos
- No puede estar desconectado (islas)
- Puede existir más de 1 por grafo
- Algoritmos:
 - Kruskal algorithm
 - Prim's algorithm
 - Dijkstra algorithm
 - Floyd-Warshall algorithm

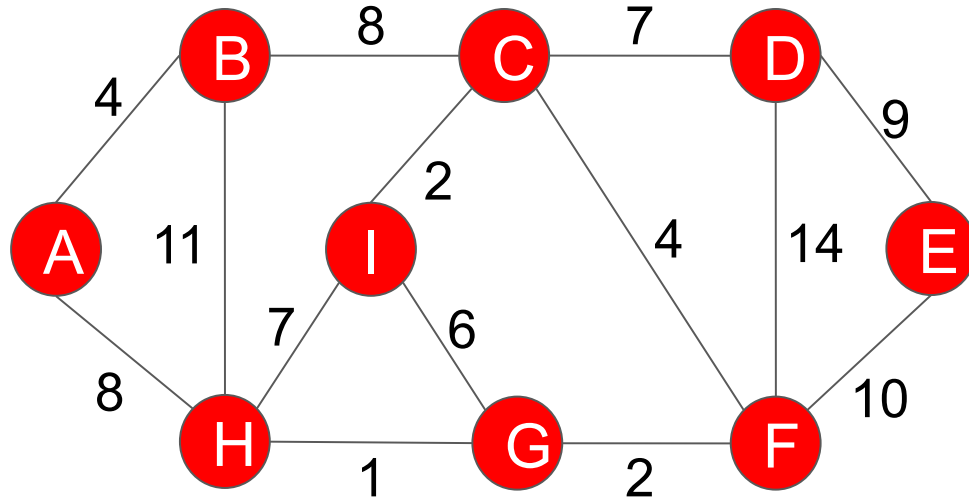
**Grafo
conexo
no dirigido**



**Árbol
recubridor
mínimo**

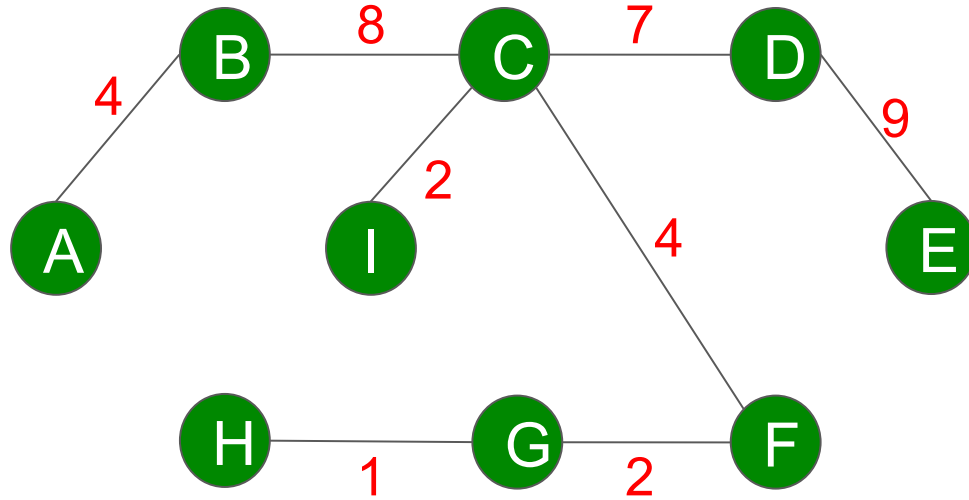


Grafos: **Árbol recubridor mínimo (Kruskal)**



1. Seleccionamos la arista de menor peso de todo el grafo
2. Repetimos 1, sin seleccionar la misma, y evitando ciclos

Grafos: Árbol recubridor mínimo (Kruskal)



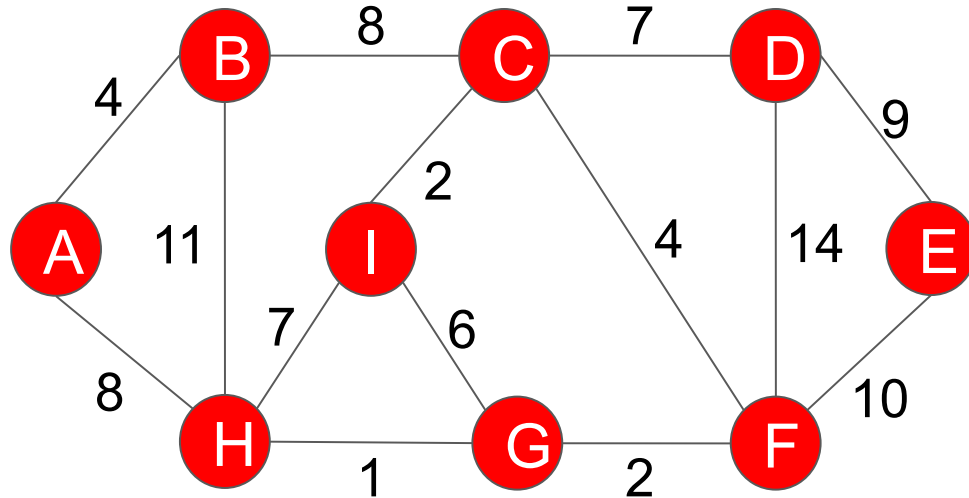
1. Seleccionamos la arista de menor peso de todo el grafo
2. Repetimos 1, sin seleccionar la misma, y evitando ciclos

PESO=37

Ejemplo: Si empezamos en H

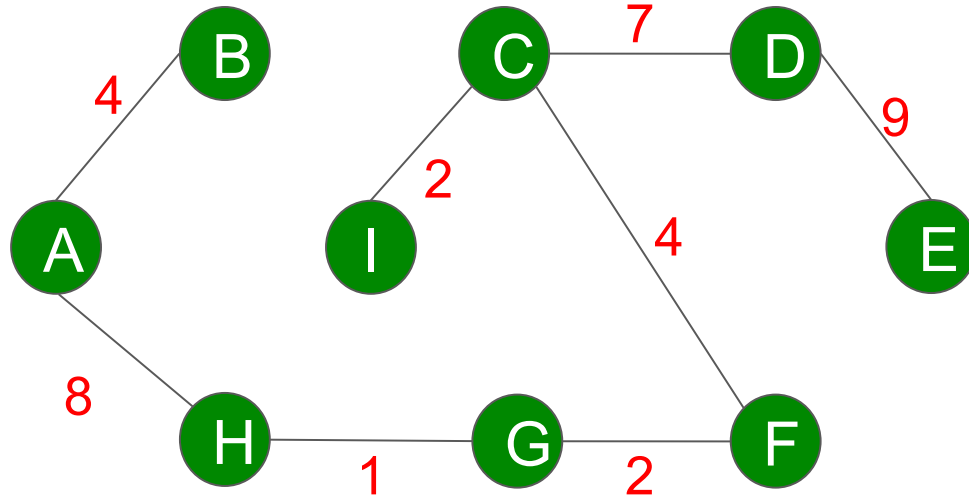
Camino: HG, GF, IC, CF, BA, ~~IG(ciclo)~~, ~~IH(ciclo)~~, CD, DE, ~~EF(ciclo)~~, ~~DF(ciclo)~~, CB, ~~BH(ciclo)~~ y ~~AH(ciclo)~~

Grafos: Árbol recubridor mínimo (Prim)



1. Seleccionamos un nodo cualquiera (con arista min)
2. Avanzamos en la arista de menor peso
3. Repetimos el Paso 2 evitando ciclos
4. Cuando se acaban las posibilidades, seleccionamos la arista de menor peso desde cualquier nodo del árbol, hasta visitar todos los nodos

Grafos: Árbol recubridor mínimo (Prim)



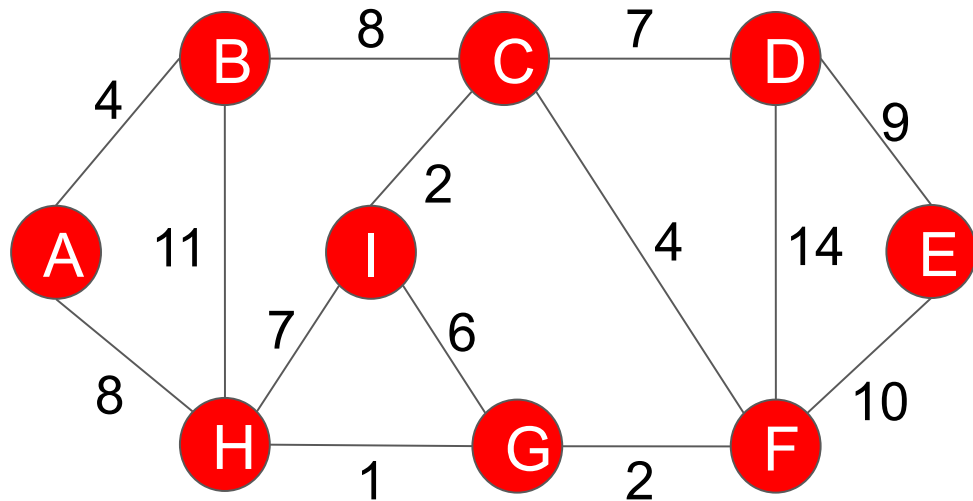
1. Seleccionamos un nodo cualquiera
2. Avanzamos en la arista de menor peso
3. Repetimos el Paso 2 evitando ciclos
4. Cuando se acaban las posibilidades, seleccionamos la arista de menor peso desde cualquier nodo del árbol, hasta visitar todos los nodos

Ejemplo: Si empezamos en H

Camino: HG, GF, FC, CI, BA, AH, DC, DE

PESO=37

Grafos: Camino más corto (Dijkstra)



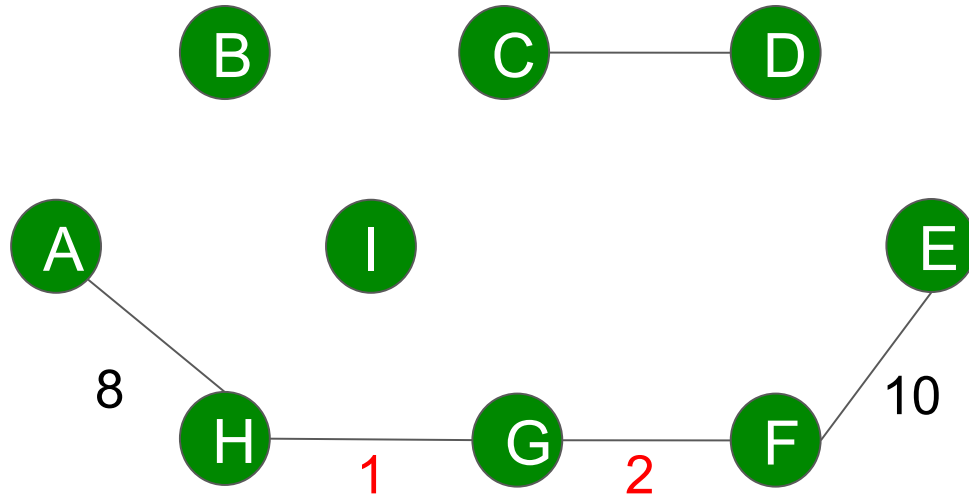
Se usa para calcular el camino mínimo entre dos vértices. Se usa una tabla de seguimiento.

Por ejemplo, para ir desde A hasta E.

[illegible]

Pasos	Vert.	A	B	C	D	E	F	G	H	I	Camino
	A	–	(4, A)	∞	∞	∞	∞	∞	(8, A)	∞	A-B
+4	B	–	–	(12, B)	∞	∞	∞	∞	(8, A) (15, B)	∞	A-H
+8	H	–	–	(12, B)	∞	∞	∞	(9, H)	–	(15, H)	A-H-G
+9	G	–	–	(12, B)	∞	∞	(11, G)	–	–	(15, H) (15, G)	A-H-G-F
+11	F	–	–	(12, B) (15, F)	(25, F)	(21, F)	–	–	–	(15, H)	A-H-G-F-G A-B-C
+12	C	–	–	–	(25, F) (19, C)	(21, F)	–	–	–	(15, H) (14, C)	A-B-C-I
+14	I	–	–	–	(19, C)	(21, F)	–	–	–	–	A-B-C-I A-B-C-D
+19	D	–	–	–	–	(21, F) (28, D)	–	–	–	–	A-H-G-F-E
+21	E										

Grafos: Reconstrucción de camino (Dijkstra)

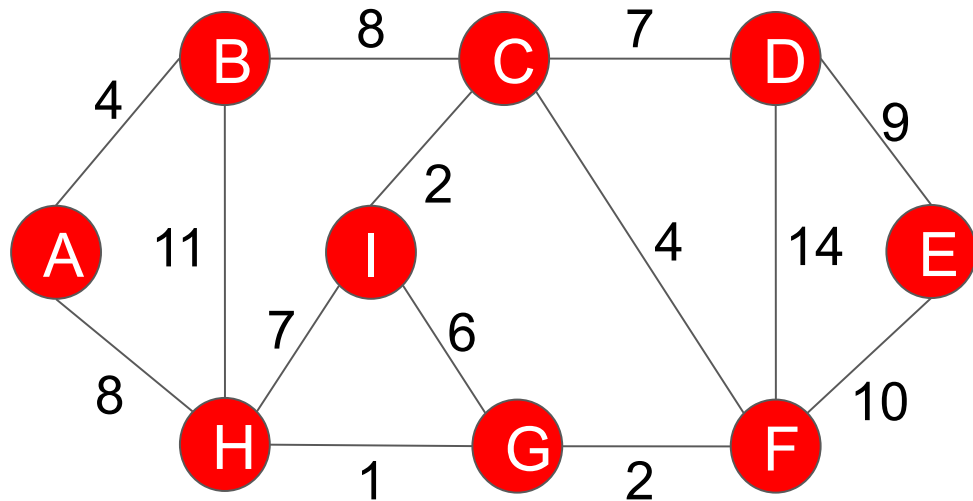


A-H-G-F-E

Ejemplo: Si empezamos en A
Camino: AH, HG, GF, FE -

Peso: 21

Grafos: Distancia más corta (Floyd-Warshall)



El algoritmo F-W usa programación dinámica para hallar la distancia más corta entre **TODOS** los pares de nodos de un grafo con pesos.

Se utilizan 2 matrices, la de costos y la de recorridos.

1. Cada nodo se usa como pivote
2. Se compara la suma de cada una de las intersecciones contra el valor actual
3. El menor de los valores prevalece en la matriz de costo y se actualiza el nombre del nodo en la matriz de recorridos con el nodo pivote

Grafos: Árbol recubridor mínimo (Floyd-Warshall)

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	∞	∞	∞	∞	∞	8	∞
B	4	0	8	∞	∞	∞	∞	11	∞
C	∞	8	0	7	∞	4	∞	∞	2
D	∞	∞	7	0	9	14	∞	∞	∞
E	∞	∞	∞	9	0	10	∞	∞	∞
F	∞	∞	4	14	10	0	2	∞	∞
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	∞	∞	∞	∞	1	0	7
I	∞	∞	2	∞	∞	∞	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	C	D	E	F	G	H	I
B	A	-	C	D	E	F	G	H	I
C	A	B	-	D	E	F	G	H	I
D	A	B	C	-	E	F	G	H	I
E	A	B	C	D	-	F	G	H	I
F	A	B	C	D	E	-	G	H	I
G	A	B	C	D	E	F	-	H	I
H	A	B	C	D	E	F	G	-	I
I	A	B	C	D	E	F	G	H	-

FW: Pivote A

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	∞	∞	∞	∞	∞	8	∞
B	4	0	8	∞	∞	∞	∞	11	∞
C	∞	8	0	7	∞	4	∞	∞	2
D	∞	∞	7	0	9	14	∞	∞	∞
E	∞	∞	∞	9	0	10	∞	∞	∞
F	∞	∞	4	14	10	0	2	∞	∞
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	∞	∞	∞	∞	1	0	7
I	∞	∞	2	∞	∞	∞	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	C	D	E	F	G	H	I
B	A	-	C	D	E	F	G	H	I
C	A	B	-	D	E	F	G	H	I
D	A	B	C	-	E	F	G	H	I
E	A	B	C	D	-	F	G	H	I
F	A	B	C	D	E	-	G	H	I
G	A	B	C	D	E	F	-	H	I
H	A	B	C	D	E	F	G	-	I
I	A	B	C	D	E	F	G	H	-

FW: Pivote B

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	∞	∞	∞	∞	8	∞
B	4	0	8	∞	∞	∞	∞	11	∞
C	12	8	0	7	∞	4	∞	19	2
D	∞	∞	7	0	9	14	∞	∞	∞
E	∞	∞	∞	9	0	10	∞	∞	∞
F	∞	∞	4	14	10	0	2	∞	∞
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	19	∞	∞	∞	1	0	7
I	∞	∞	2	∞	∞	∞	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	D	E	F	G	H	I
B	A	-	C	D	E	F	G	H	I
C	B	B	-	D	E	F	G	B	I
D	A	B	C	-	E	F	G	H	I
E	A	B	C	D	-	F	G	H	I
F	A	B	C	D	E	-	G	H	I
G	A	B	C	D	E	F	-	H	I
H	A	B	B	D	E	F	G	-	I
I	A	B	C	D	E	F	G	H	-

FW: Pivote C

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	∞	16	∞	8	14
B	4	0	8	15	∞	12	∞	11	10
C	12	8	0	7	∞	4	∞	19	2
D	19	15	7	0	9	11	∞	26	9
E	∞	∞	∞	9	0	10	∞	∞	∞
F	16	12	4	11	10	0	2	23	6
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	19	26	∞	23	1	0	7
I	14	10	2	9	∞	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	E	C	G	H	C
B	A	-	C	C	E	C	G	H	C
C	B	B	-	D	E	F	G	B	I
D	C	C	C	-	E	C	G	C	C
E	A	B	C	D	-	F	G	H	I
F	C	C	C	C	E	-	G	C	C
G	A	B	C	D	E	F	-	H	I
H	A	B	B	C	E	C	G	-	I
I	C	C	C	C	E	C	G	H	-

FW: Pivote D

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	28	16	∞	8	14
B	4	0	8	15	24	12	∞	11	10
C	12	8	0	7	16	4	∞	19	2
D	19	15	7	0	9	11	∞	26	9
E	28	24	16	9	0	10	∞	35	18
F	16	12	4	11	10	0	2	23	6
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	19	26	35	23	1	0	7
I	14	10	2	9	18	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	D	C	G	H	C
B	A	-	C	C	D	C	G	H	C
C	B	B	-	D	D	F	G	B	I
D	C	C	C	-	E	C	G	C	C
E	D	D	D	D	-	F	G	D	D
F	C	C	C	C	E	-	G	C	C
G	A	B	C	D	E	F	-	H	I
H	A	B	B	C	D	C	G	-	I
I	C	C	C	C	D	C	G	H	-

FW: Pivote E

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	28	16	∞	8	14
B	4	0	8	15	24	12	∞	11	10
C	12	8	0	7	16	4	∞	19	2
D	19	15	7	0	9	11	∞	26	9
E	28	24	16	9	0	10	∞	35	18
F	16	12	4	11	10	0	2	23	6
G	∞	∞	∞	∞	∞	2	0	1	6
H	8	11	19	26	35	23	1	0	7
I	14	10	2	9	18	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	D	C	G	H	C
B	A	-	C	C	D	C	G	H	C
C	B	B	-	D	D	F	G	B	I
D	C	C	C	-	E	C	G	C	C
E	D	D	D	D	-	F	G	D	D
F	C	C	C	C	E	-	G	C	C
G	A	B	C	D	E	F	-	H	I
H	A	B	B	C	D	C	G	-	I
I	C	C	C	C	D	C	G	H	-

FW: Pivote F

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	26	16	18	8	14
B	4	0	8	15	22	12	14	11	10
C	12	8	0	7	14	4	6	19	2
D	19	15	7	0	9	11	13	26	9
E	26	22	14	9	0	10	12	33	16
F	16	12	4	11	10	0	2	23	6
G	18	14	6	13	12	2	0	1	6
H	8	11	19	26	33	23	1	0	7
I	14	10	2	9	16	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	F	C	F	H	C
B	A	-	C	C	F	C	F	H	C
C	B	B	-	D	F	F	F	B	I
D	C	C	C	-	E	C	F	C	C
E	F	F	F	D	-	F	F	F	F
F	C	C	C	C	E	-	G	C	C
G	F	F	F	F	F	F	-	H	I
H	A	B	B	C	F	C	G	-	I
I	C	C	C	C	F	C	G	H	-

FW: Pivote G

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	26	16	18	8	14
B	4	0	8	15	22	12	14	11	10
C	12	8	0	7	14	4	6	7	2
D	19	15	7	0	9	11	13	14	9
E	26	22	14	9	0	10	12	13	16
F	16	12	4	11	10	0	2	3	6
G	18	14	6	13	12	2	0	1	6
H	8	11	7	14	13	3	1	0	7
I	14	10	2	9	16	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	F	C	F	H	C
B	A	-	C	C	F	C	F	H	C
C	B	B	-	D	F	F	F	G	I
D	C	C	C	-	E	C	F	G	C
E	F	F	F	D	-	F	F	G	F
F	C	C	C	C	E	-	G	G	C
G	F	F	F	F	F	F	-	H	I
H	A	B	G	G	G	G	G	-	I
I	C	C	C	C	F	C	G	H	-

FW: Pivote H

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	21	11	9	8	14
B	4	0	8	15	22	12	12	11	10
C	12	8	0	7	14	4	6	7	2
D	19	15	7	0	9	11	13	14	9
E	21	22	14	9	0	10	12	13	16
F	11	12	4	11	10	0	2	3	6
G	9	12	6	13	12	2	0	1	6
H	8	11	7	14	13	3	1	0	7
I	14	10	2	9	16	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	H	H	H	H	C
B	A	-	C	C	F	C	H	H	C
C	B	B	-	D	F	F	F	G	I
D	C	C	C	-	E	C	F	G	C
E	H	F	F	D	-	F	F	G	F
F	H	C	C	C	E	-	G	G	C
G	H	H	F	F	F	F	-	H	I
H	A	B	G	G	G	G	G	-	I
I	C	C	C	C	F	C	G	H	-

FW: Pivote I

A-E: **A-H-G-F-E**

A-D: **A-B-C-D**

Matriz de adyacencia (pesos)

	A	B	C	D	E	F	G	H	I
A	0	4	12	19	21	11	9	8	14
B	4	0	8	15	22	12	12	11	10
C	12	8	0	7	14	4	6	7	2
D	19	15	7	0	9	11	13	14	9
E	21	22	14	9	0	10	12	13	16
F	11	12	4	11	10	0	2	3	6
G	9	12	6	13	12	2	0	1	6
H	8	11	7	14	13	3	1	0	7
I	14	10	2	9	16	6	6	7	0

Matriz de adyacencia (recorrido)

	A	B	C	D	E	F	G	H	I
A	-	B	B	C	H	H	H	H	C
B	A	-	C	C	F	C	H	H	C
C	B	B	-	D	F	F	F	G	I
D	C	C	C	-	E	C	F	G	C
E	H	F	F	D	-	F	F	G	F
F	H	C	C	C	E	-	G	G	C
G	H	H	F	F	F	F	-	H	I
H	A	B	G	G	G	G	G	-	I
I	C	C	C	C	F	C	G	H	-



Tarea

1. Implementar en Java los algoritmos
 - a. Kruskal algorithm
 - b. Prim's algorithm
 - c. Dijkstra algorithm
 - d. Floyd-Warshall algorithm